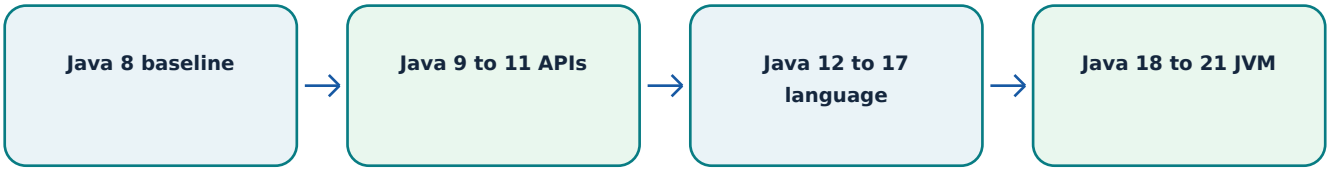
JAVA 8 TO JAVA 21

The visual, code-first, beginner-to-advanced interactive textbook

Every release from Java 9 through Java 21. Learn the problem before the solution. Predict before reading the answer. Code, diagrams, tradeoffs, and interview reasoning in every lesson.



How to use this book

Each lesson has three deliberately separate learning pages: (1) the problem and old code, (2) the modern code, diagram and caveats, and (3) tricky interview questions with explained answers plus hands-on practice. Stop after a prediction, run the code, then check the answer.

Version labels are essential. Unless otherwise stated, code examples target JDK 21 and show when the feature first became available. Preview examples are labelled and may not compile with stable Java 21 syntax.

Concepts you should know before you begin

Term	Plain-English meaning
Reference	A value that points to an object; copying it does not clone the object.
Mutable	Can change after creation.
Unmodifiable	You cannot add, remove or replace elements through that collection.
Snapshot	Membership captured at one moment.
Shallow copy	New container, same referenced element objects.
JVM	The program that runs Java bytecode.
Thread	A path of execution within a program.
Preview	Feature offered for feedback, not finalized.

Version map: what changed and when

Java 9: List.of, Set.of and Map.of, Stream takeWhile, dropWhile, ofNullable, Optional additions, Private interface methods, Java Platform Module System (JPMS), JShell and try-with-resources improvement

Java 10: var local variable inference, List.copyOf and unmodifiable collectors

Java 11: String and Files convenience APIs, HTTP Client, Optional.isEmpty, Predicate.not, source launcher

Java 12: Switch expressions preview

Java 13: Text blocks preview

Java 14: Final switch expressions, Helpful NullPointerExceptions

Java 15: Final text blocks, Hidden classes and JVM context

Java 16: Records, Pattern matching instanceof, Stream.toList

Java 17: Sealed classes, Strong encapsulation and migration

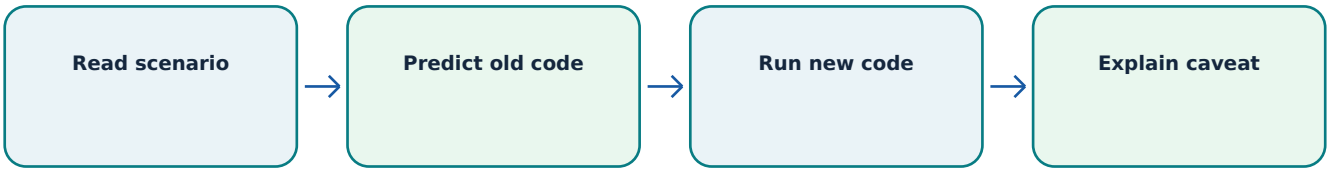
Java 18: UTF-8 default charset, Simple Web Server and Javadoc snippets

Java 19: Virtual threads first preview, Record patterns first preview

Java 20: Structured concurrency and scoped values previews, Virtual threads and record patterns second previews

Java 21: Virtual threads finalized, Pattern matching for switch finalized, Record patterns finalized, Sequenced collections, Generational ZGC and garbage collection, Java 21 preview features, Migration checklist and compatibility

A practical learning loop



Do not memorize an API until you can answer: What did developers do before it? What problem remains? What does the new code change? What does it NOT change?

Use javac --release 21 for stable examples. For release-history exercises, verify API availability against the stated version. Some examples are illustrative fragments requiring imports, helper methods, or surrounding classes; these are explicitly introduced as fragments.

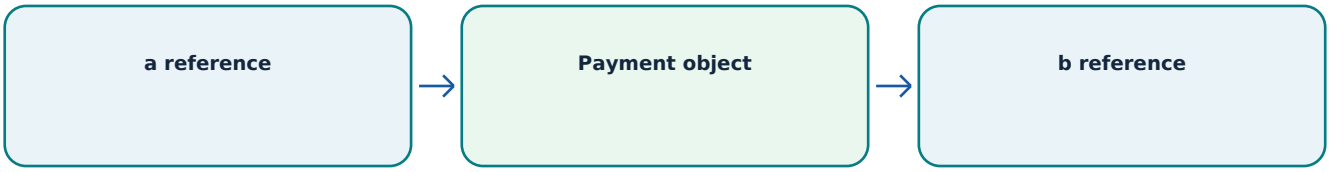
Foundation 1: References vs copies

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
Payment a = new Payment("UPI");
Payment b = a;
b.method = "CARD";
// a.method is CARD
```

Two variables can point to the same object. Changing object state through either variable affects the one shared object. Reassigning b to new Payment("CASH") does not change a.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

Foundation 2: final vs unmodifiable

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
final List<String> xs = new ArrayList<>();
xs.add("UPI"); // allowed
// xs = new ArrayList<>(); // compile error
```

final locks the variable to its reference value; it does not freeze the object. A final ArrayList is still mutable.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

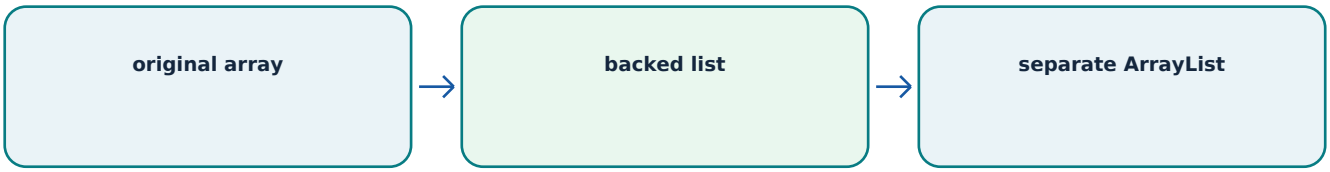
Foundation 3: Arrays.asList vs ArrayList copy

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
String[] a = {"Java", "Python"};
List<String> view = Arrays.asList(a);
List<String> copy = new ArrayList<>(view);
a[0] = "Go";
// view: [Go, Python]
// copy: [Java, Python]
```

Arrays.asList(array) is backed by the caller array. new ArrayList<>(...) makes another container. Neither deep-copies mutable element objects.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

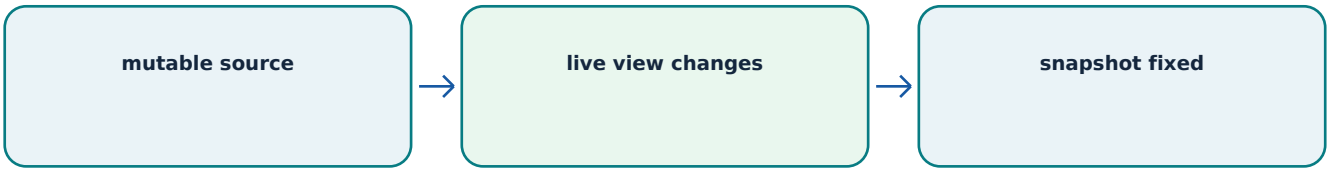
Foundation 4: Read-only live view vs snapshot

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
List<String> src = new ArrayList<>(List.of("A"));
List<String> live = Collections.unmodifiableList(src);
List<String> snap = List.copyOf(src);
src.add("B");
// live: [A, B]; snap: [A]
```

The wrapper blocks mutation through live but reads the original list. copyOf captures membership separately.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

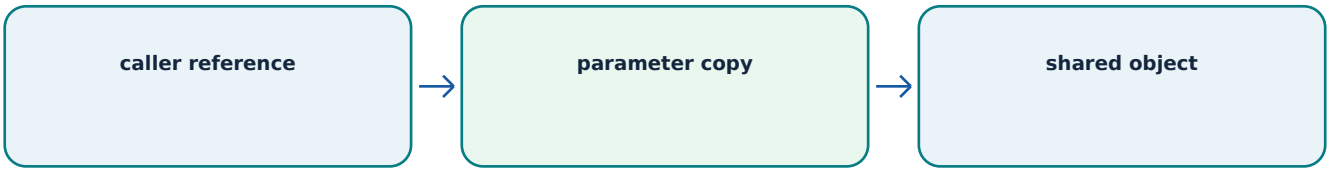
Foundation 5: Pass-by-value with references

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
static void f(Payment p) {
  p.method = "CARD";
  p = new Payment("CASH");
}
// caller sees CARD, not CASH
```

Java copies the reference value into parameter p. Both initially point to one object; reassignment of p changes only p.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

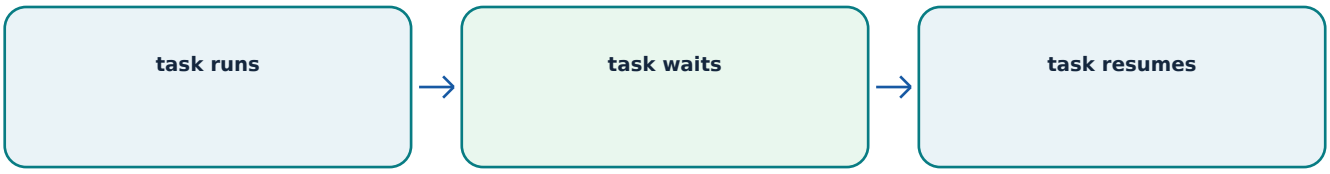
Foundation 6: Threads and blocking

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
Thread t = new Thread(() -> {
  try { Thread.sleep(100); }
  catch (InterruptedException e) {
    Thread.currentThread().interrupt();
  }
});
t.start();
```

A thread executes instructions. sleep makes it wait. An executor manages tasks and threads. Virtual threads help when many tasks spend time waiting, not when all are calculating.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

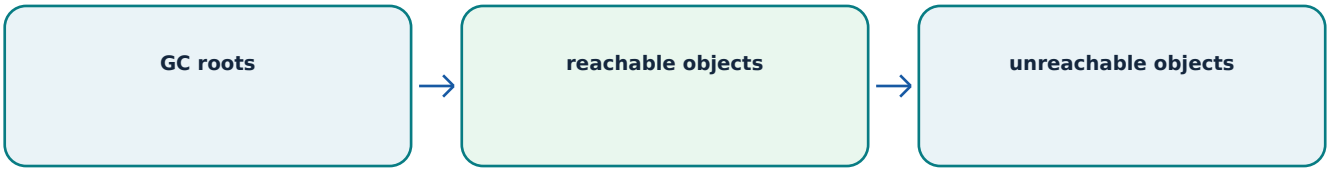
Foundation 7: JVM, heap, reachability

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
Object a = new Object();
Object b = a;
a = null;
// object is still reachable through b
```

Objects commonly live in heap memory. Garbage collection reclaims objects that are no longer reachable from GC roots. Clearing one of several references is not enough.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

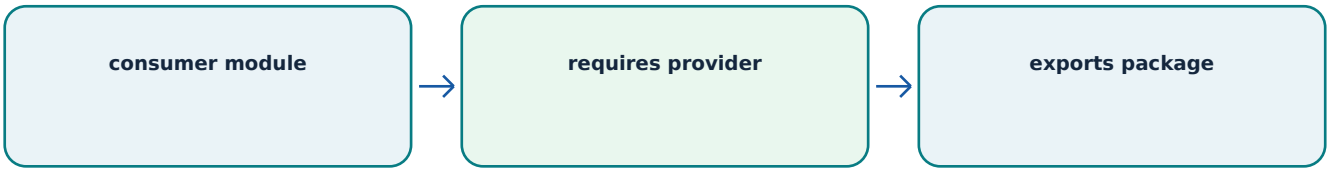
Foundation 8: Modules and package boundaries

Why this matters: Many modern Java features only make sense when the older behavior is clear. This page explains the prerequisite before using it in later lessons.

Run and reason about this code

```
// module payment.api { exports payment.api; }
// module payment.app { requires payment.api; }
```

A package groups related classes; a module groups packages and explicitly states dependencies and exported packages.



Pause and predict

Before running the code, identify how many distinct objects exist and which variables refer to each. Then change one line and predict the result.

Interview check

Explain the difference between changing an object and changing a variable that references it. Use the code above, not a memorized definition.

LESSON 01 | JAVA 9 | List.of, Set.of and Map.of

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A payment system has approved methods that must not change when another class gets the list. The old code accidentally returns the original mutable ArrayList, so any caller can add an unsupported method.

How developers approached it before

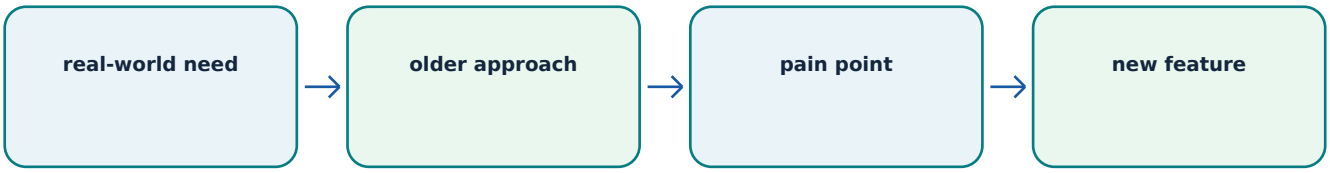
```
List<String> payments = new ArrayList<>();
payments.add("UPI");
payments.add("CARD");
List<String> shared = payments;
shared.add("CASH"); // original also changes
```

What is inconvenient or risky?

Factories produce compact unmodifiable collections.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 01 | JAVA 9 | List.of, Set.of and Map.of

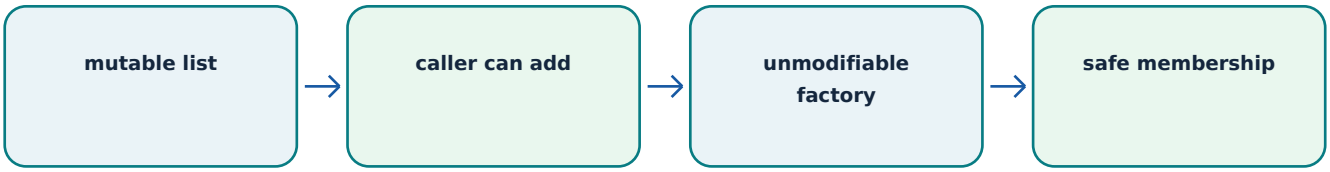
Modern approach (see version notes)

```
List<String> methods = List.of("UPI", "CARD");
Set<String> unique = Set.of("UPI", "CARD");
Map<String, Integer> fees = Map.of("UPI", 0, "CARD", 2);
// methods.add("CASH"); // throws
```

Why this helps

Factories produce compact unmodifiable collections. List keeps duplicates and order; Set rejects duplicate inputs; Map rejects duplicate keys. They reject nulls. They do not freeze mutable objects stored inside.

Follow the execution visually



Cons, caveats and common mistakes

1. List.of(array) does not use the caller array as backing storage.
2. List.of(new StringBuilder("x")) still contains a mutable element.
3. Set.of("A","A") throws IllegalArgumentException; List.of("A","A") works.

Which approach should you choose?

Approach	Behavior	Good use case
ArrayList	mutable; add/remove/set	dynamic state
Arrays.asList	fixed-size; set allowed	array-backed view
unmodifiableList	read-only live view	read-only current state
List.of	unmodifiable, null-free	fixed values

LESSON 01 | JAVA 9 | List.of, Set.of and Map.of

Frequently asked and tricky interview questions - with reasoning

Q1. Does final List make the list unmodifiable?

Answer and reasoning: No. final stops reference reassignment; add() can still mutate an ArrayList.

Q2. What happens with List.of("A", null)?

Answer and reasoning: NullPointerException during construction.

Q3. Can you change an Employee stored in List.of?

Answer and reasoning: Yes, if Employee itself is mutable; the list only blocks add/remove/set.

Practice: beginner -> intermediate -> advanced

Level 1: Predict: List.of("A","B").set(0,"C")

Level 2: Try Set.of("A","A") and explain the exception.

Level 3: Create a fixed configuration and a mutable shopping cart; justify each choice.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 9 milestone: List.of, Set.of and Map.of. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | List.of, Set.of, Map.of

Lesson 1 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why do fixed configuration values accidentally change? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
List<String> x = new ArrayList<>();
x.add("UPI"); x.add("CARD");
List<String> view = Collections.unmodifiableList(x);
```

AFTER: WHAT WE CAN WRITE

```
List<String> fixed = List.of("UPI", "CARD");
Set<String> unique = Set.of("UPI", "CARD");
Map<String,Integer> fees = Map.of("UPI", 0);
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

A view still follows changes to x; List.of has no mutable source collection. The elements themselves can still be mutable.

INTERVIEW TRAP - PREDICT FIRST

What does List.of("UPI", null) do?

ANSWER AND REASONING

It throws NullPointerException immediately. List.of rejects null values.

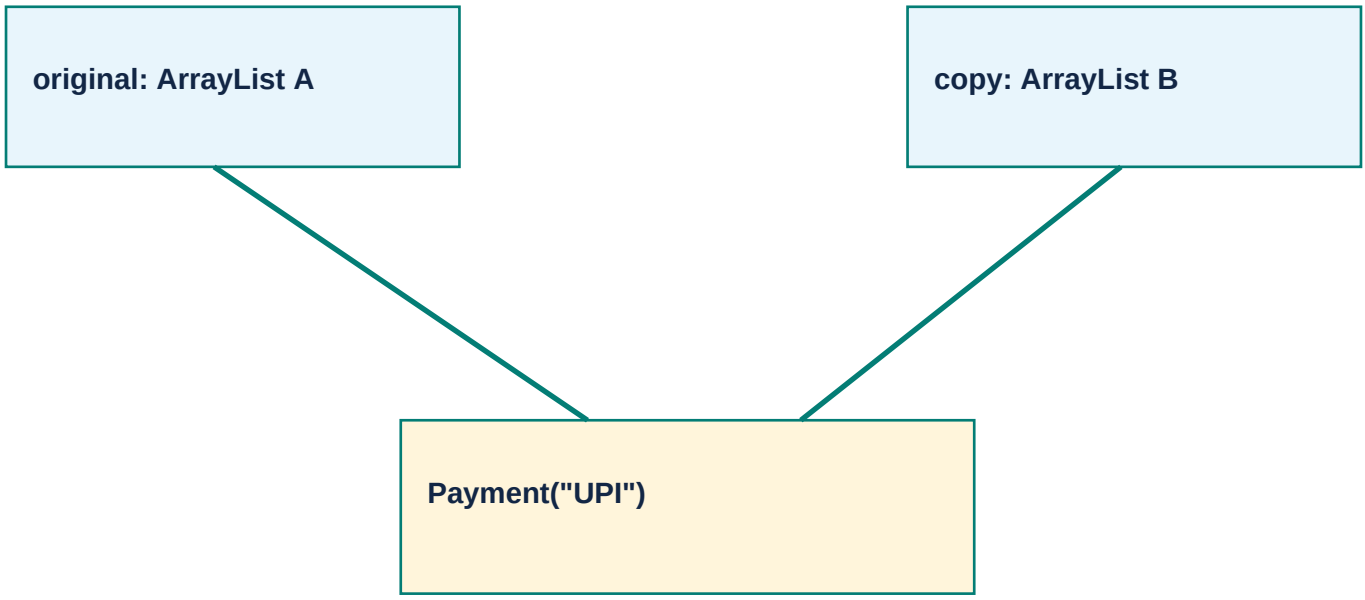
TRY IT / WORKED SOLUTION

Try List.of("A","A") and Set.of("A","A"). Answer: List accepts duplicates; Set.of throws IllegalArgumentException for duplicate arguments.

MEMORY LAB | Views, snapshots and shared objects

Read the arrows as references or execution steps; run the code before checking the answer.

ONE PAYMENT OBJECT, TWO DIFFERENT LIST CONTAINERS



CODE: SHARED OBJECT, SEPARATE MEMBERSHIP

```
Payment p = new Payment("UPI");
List<Payment> a = new ArrayList<>(); a.add(p);
List<Payment> b = List.copyOf(a);
p.method = "CARD"; a.add(new Payment("CASH"));
// b.get(0).method = CARD; b.size() = 1
```

WHY THIS MATTERS

copyOf copies element references, not Payment objects. A view (unmodifiableList(a)) would also see the later CASH membership; a copy does not.

LESSON 02 | JAVA 9 | Stream takeWhile, dropWhile, ofNullable

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A sorted list of transactions is being read until the first transaction older than a cutoff. A Java 8 filter keeps scanning and does not express the stop-at-first-failure rule.

How developers approached it before

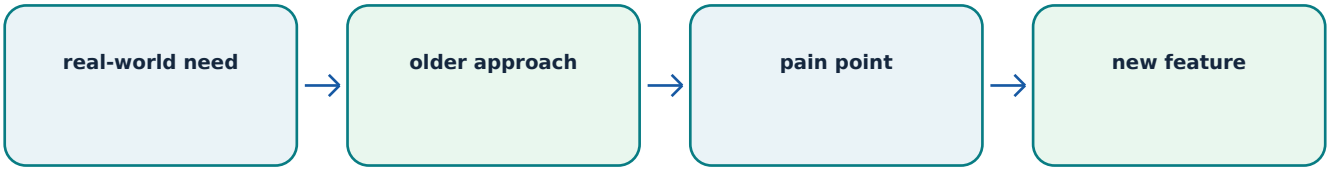
```
List<Integer> values = Arrays.asList(2, 3, 9, 1);
values.stream().filter(x -> x < 5)
      .forEach(System.out::println); // 2,3,1
```

What is inconvenient or risky?

filter tests each item; takeWhile takes the longest matching prefix in encounter order; dropWhile skips that prefix.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 02 | JAVA 9 | Stream takeWhile, dropWhile, ofNullable

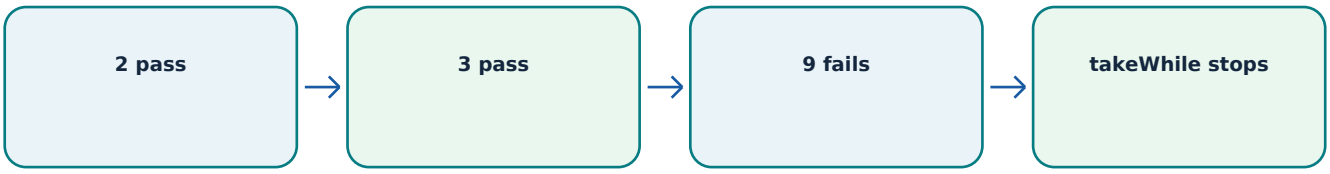
Modern approach (see version notes)

```
List<Integer> values = List.of(2, 3, 9, 1);
values.stream().takeWhile(x -> x < 5)
      .forEach(System.out::println); // 2,3
values.stream().dropWhile(x -> x < 5)
      .forEach(System.out::println); // 9,1
Stream.ofNullable(null).count(); // 0
```

Why this helps

filter tests each item; takeWhile takes the longest matching prefix in encounter order; dropWhile skips that prefix. This matters when the order is meaningful.

Follow the execution visually



Cons, caveats and common mistakes

1. On unordered streams, do not assume a particular prefix.
2. takeWhile is not the same as filter.
3. ofNullable(x) creates zero or one stream element.

Which approach should you choose?

Approach	Behavior	Good use case
filter	test all candidates	non-prefix selection
takeWhile	keep matching prefix	ordered threshold
dropWhile	remove matching prefix	skip leading items

LESSON 02 | JAVA 9 | Stream takeWhile, dropWhile, ofNullable

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Why does takeWhile skip 1 after 9?

Answer and reasoning: It stops at the first failure, unlike filter.

Q2. What does Stream.ofNullable(null).count() return?

Answer and reasoning: 0, because no element is produced.

Q3. Does takeWhile always improve performance?

Answer and reasoning: No; benefits depend on order, source and pipeline.

Practice: beginner -> intermediate -> advanced

Level 1: Compare filter and takeWhile on [2,3,9,1].

Level 2: Use dropWhile to skip a header prefix.

Level 3: Explain ordered versus unordered sources.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 9 milestone: Stream takeWhile, dropWhile, ofNullable. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Stream takeWhile / dropWhile / ofNullable

Lesson 1 | Code examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

When does stopping at the first failure differ from filtering every item? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
List<Integer> a = Arrays.asList(2, 4, 9, 3);
a.stream().filter(n -> n < 5)
 .forEach(System.out::println);
```

AFTER: WHAT WE CAN WRITE

```
List<Integer> a = List.of(2, 4, 9, 3);
a.stream().takeWhile(n -> n < 5)
 .forEach(System.out::println);
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

filter checks every element; takeWhile stops at the first failed predicate on an ordered stream.

INTERVIEW TRAP - PREDICT FIRST

What prints with takeWhile above?

ANSWER AND REASONING

2 and 4. The 9 stops the prefix, so 3 is never included.

TRY IT / WORKED SOLUTION

Replace takeWhile with dropWhile. Answer: dropWhile removes the initial 2 and 4, then retains 9 and 3.

LESSON 03 | JAVA 9 | Optional additions

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A repository might not find a customer. Java 8 Optional can express absence, but a fallback Optional and a branch for both cases require extra code.

How developers approached it before

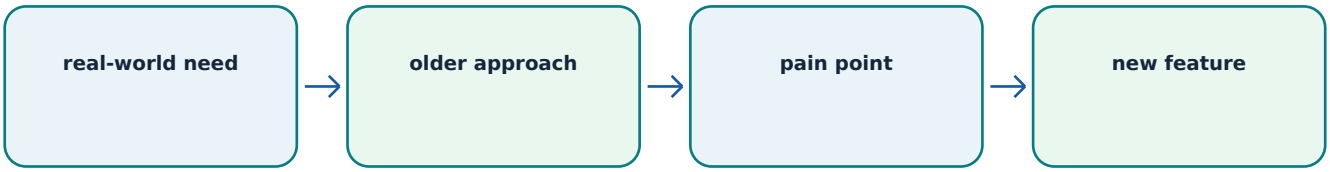
```
Optional<String> name = Optional.of("Ada");
if (name.isPresent()) System.out.println(name.get());
else System.out.println("missing");
```

What is inconvenient or risky?

ifPresentOrElse handles both branches; or supplies another Optional lazily; stream turns an Optional into zero or one stream elements.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 03 | JAVA 9 | Optional additions

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

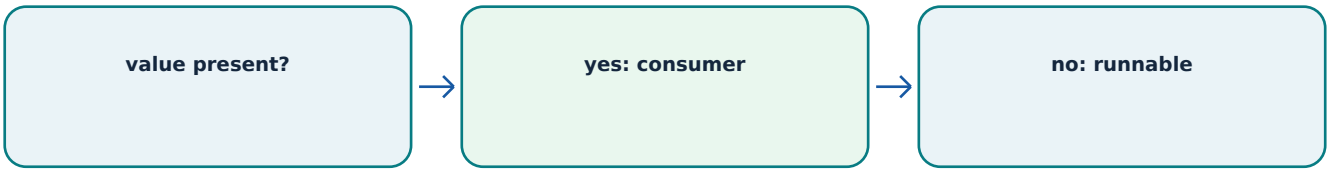
Modern approach (see version notes)

```
Optional<String> name = Optional.of("Ada");
name.ifPresentOrElse(System.out::println,
    () -> System.out.println("missing"));
Optional<String> result = name.or(() -> Optional.of("guest"));
long count = name.stream().count();
```

Why this helps

ifPresentOrElse handles both branches; or supplies another Optional lazily; stream turns an Optional into zero or one stream elements.

Follow the execution visually



Cons, caveats and common mistakes

1. Optional.or takes Supplier<Optional<T>>, not Supplier<T>.
2. Avoid get() without presence checks.
3. Optional is not a replacement for every nullable field.

Which approach should you choose?

Approach	Behavior	Good use case
ifPresentOrElse	two branches	display / action
or	fallback Optional	lookup chain
stream	0-or-1 elements	pipeline flattening

LESSON 03 | JAVA 9 | Optional additions

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Is orElseGet lazy?

Answer and reasoning: Yes; supplier runs only if value is absent.

Q2. What does Optional.empty().stream().count() return?

Answer and reasoning: 0.

Q3. Does Optional.of(null) work?

Answer and reasoning: No, it throws NullPointerException; use ofNullable.

Practice: beginner -> intermediate -> advanced

Level 1: Implement an optional fallback without get().

Level 2: Turn List<Optional<String>> into Stream<String>.

Level 3: Compare orElse and orElseGet side effects.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 9 milestone: Optional additions. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Optional additions

Lesson 3 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can we handle a missing value without repeated if checks? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
Optional<String> o = Optional.empty();
if (o.isPresent()) use(o.get());
else logMissing();
```

AFTER: WHAT WE CAN WRITE

```
Optional<String> o = Optional.empty();
o.ifPresentOrElse(v -> use(v),
    () -> logMissing());
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Optional.or supplies another Optional lazily; Optional.stream turns zero-or-one values into a stream.

INTERVIEW TRAP - PREDICT FIRST

Does orElse(expensive()) call expensive when value exists?

ANSWER AND REASONING

Yes. The argument is evaluated first. orElseGet(() -> expensive()) is lazy.

TRY IT / WORKED SOLUTION

Compare orElse and orElseGet with a printed supplier. Answer: Only orElseGet skips supplier work when the Optional contains a value.

LESSON 04 | JAVA 9 | Private interface methods

Real-world problem, in plain language

Two default methods in an interface repeat validation code. Before Java 9, an interface could not place the shared helper in a private method.

How developers approached it before

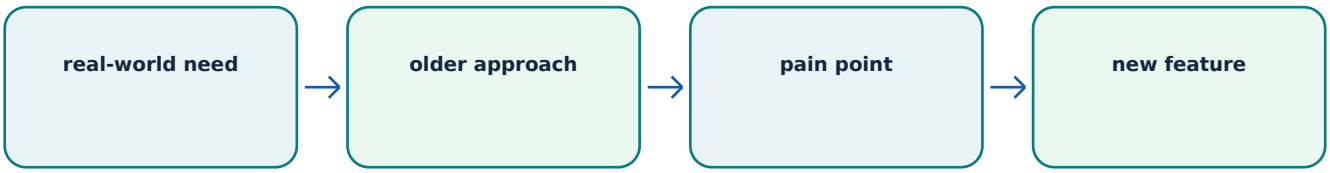
```
interface Rules {
  default boolean validA(String x) {
    return x != null && !x.isEmpty();
  }
  default boolean validB(String x) {
    return x != null && !x.isEmpty();
  }
}
```

What is inconvenient or risky?

A private interface method shares internal logic without adding a method that implementing classes can call or override.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 04 | JAVA 9 | Private interface methods

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

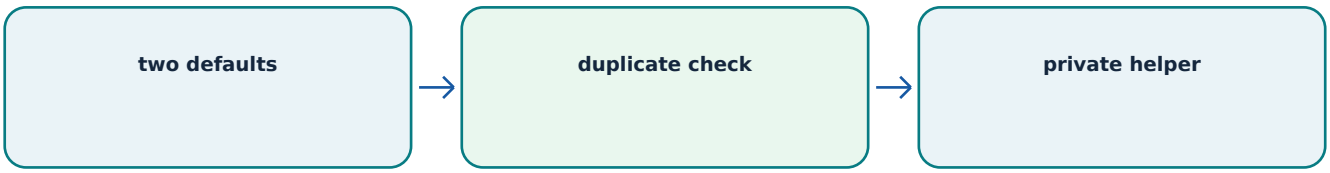
Modern approach (see version notes)

```
interface Rules {
  default boolean validA(String x) { return check(x); }
  default boolean validB(String x) { return check(x); }
  private boolean check(String x) {
    return x != null && !x.isEmpty();
  }
}
```

Why this helps

A private interface method shares internal logic without adding a method that implementing classes can call or override.

Follow the execution visually



Cons, caveats and common mistakes

1. Private methods are not inherited by implementing classes.
2. Private static methods can support static interface methods.
3. Do not use interface helpers to hide an overly complex design.

Which approach should you choose?

Approach	Behavior	Good use case
default	public behavior	implementation extension
private	internal reuse	no public API

LESSON 04 | JAVA 9 | Private interface methods

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can an implementing class call check()?

Answer and reasoning: No; it is private to the interface.

Q2. Can an interface have private static methods?

Answer and reasoning: Yes, starting in Java 9.

Q3. Does private mean the method can be overridden?

Answer and reasoning: No; it is not accessible to implementers.

Practice: beginner -> intermediate -> advanced

Level 1: Extract repeated validation into a private helper.

Level 2: Try to call the helper from a class and inspect compiler error.

Level 3: Compare interface default and private methods.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 9 milestone: Private interface methods. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Private interface methods

Lesson 4 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

What if two default interface methods share the same helper logic? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
interface Audit {
 default void start() { System.out.println("audit"); }
 default void stop() { System.out.println("audit"); }
}
```

AFTER: WHAT WE CAN WRITE

```
interface Audit {
 private void log() { System.out.println("audit"); }
 default void start() { log(); }
 default void stop() { log(); }
}
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Private interface methods reduce duplication but are not inherited or callable by implementing classes.

INTERVIEW TRAP - PREDICT FIRST

Can a class call Audit.super.log()?

ANSWER AND REASONING

No. log is private to the interface.

TRY IT / WORKED SOLUTION

Make log static and call it from default methods. Answer: A private static helper can be called by name within the interface.

LESSON 05 | JAVA 9 | Java Platform Module System (JPMS)

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A large application accidentally uses internal packages of another library. Classpath visibility makes dependency boundaries hard to state explicitly. Modules declare what is required and what is exposed.

How developers approached it before

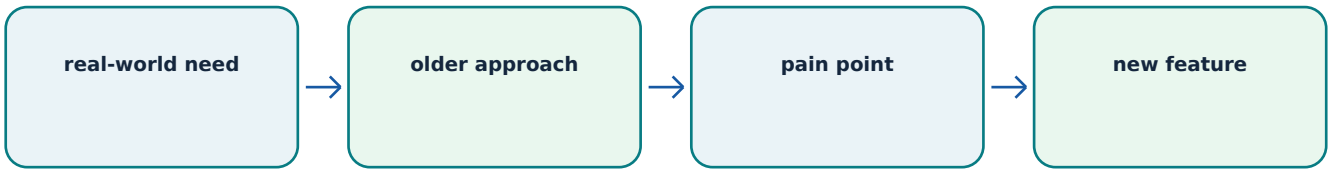
```
// Classpath project: any public class on the
// classpath may be visible to other code.
// No module-info.java declaration.
```

What is inconvenient or risky?

requires declares a module dependency; exports makes a package accessible to other modules.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 05 | JAVA 9 | Java Platform Module System (JPMS)

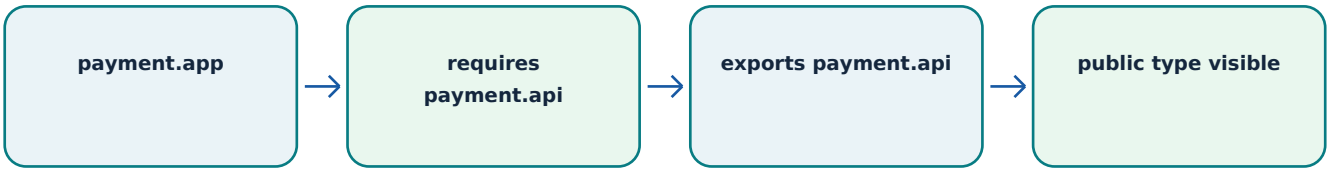
Modern approach (see version notes)

```
// payment.api/module-info.java
module payment.api { exports payment.api; }
// payment.app/module-info.java
module payment.app { requires payment.api; }
// payment.api/payment/api/Payment.java
package payment.api;
public record Payment(String id) {}
```

Why this helps

requires declares a module dependency; exports makes a package accessible to other modules. Modules are optional for many classpath applications.

Follow the execution visually



Cons, caveats and common mistakes

1. Module names and package names are different concepts.
2. exports is not the same as opens for deep reflection.
3. Migration can break code relying on internal JDK APIs.

Which approach should you choose?

Approach	Behavior	Good use case
requires	consumer dependency	app -> API
exports	public package access	API -> app
opens	reflective access	framework integration

LESSON 05 | JAVA 9 | Java Platform Module System (JPMS)

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does requires make every package public?

Answer and reasoning: No; the provider must export the package.

Q2. Can a Java 21 application still use classpath?

Answer and reasoning: Yes; modularization is not mandatory.

Q3. Why might frameworks need opens?

Answer and reasoning: Reflective access to non-public members may require an open package.

Practice: beginner -> intermediate -> advanced

Level 1: Make two modules and remove exports; explain compiler failure.

Level 2: Compare classpath and module path.

Level 3: Explain requires vs exports vs opens with one example.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 9 milestone: Java Platform Module System (JPMS). Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | JPMS modules

Lesson 5 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we stop an application from accessing implementation packages accidentally? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// classpath: public classes can be accessed
// when their packages are visible
```

AFTER: WHAT WE CAN WRITE

```
// module payment.api
module payment.api { exports payment.api; }
// module checkout.app
module checkout.app { requires payment.api; }
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

exports controls package visibility across modules; requires declares a dependency. Classpath and module path are different modes.

INTERVIEW TRAP - PREDICT FIRST

If payment.api is not exported, can another module import its public class?

ANSWER AND REASONING

No: public alone does not override module exports.

TRY IT / WORKED SOLUTION

Remove requires from checkout.app and compile. Answer: Compilation fails because the dependency is not declared.

MODULE LAB | Two modules you can compile

Read the arrows as references or execution steps; run the code before checking the answer.

PROJECT TREE

```
src/payment.api/module-info.java
src/payment.api/payment/api/Payment.java
src/checkout.app/module-info.java
src/checkout.app/checkout/Main.java
```

MODULE DESCRIPTORS

```
// payment.api/module-info.java
module payment.api { exports payment.api; }
// checkout.app/module-info.java
module checkout.app { requires payment.api; }
```

COMPILE AND RUN

```
javac -d out --module-source-path src \n  $(find src -name "*.java")
java --module-path out -m checkout.app/checkout.Main
```

FAILURE EXPERIMENT

Remove exports: importing payment.api.Payment fails even when the class is public. Remove requires: the checkout module cannot read payment.api.

LESSON 06 | JAVA 9 | JShell and try-with-resources improvement

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

You want to test a small expression without creating a full class. Separately, a resource is already stored in a final or effectively final variable, and Java 8 requires redeclaring it in try-with-resources.

How developers approached it before

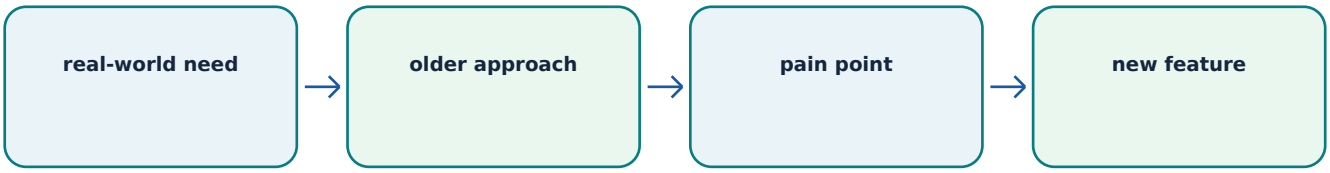
```
// Java 8 resource pattern
try (BufferedReader r = new BufferedReader(
    new StringReader("hi"))) {
  System.out.println(r.readLine());
}
```

What is inconvenient or risky?

JShell is a REPL: read, evaluate, print, loop.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 06 | JAVA 9 | JShell and try-with-resources improvement

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

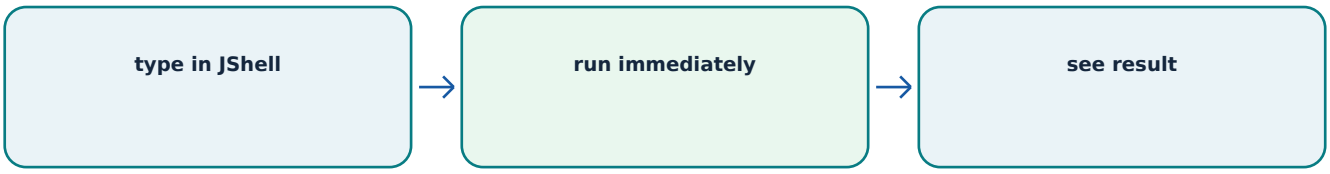
Modern approach (see version notes)

```
// Java 9: existing effectively-final resource
BufferedReader r = new BufferedReader(
    new StringReader("hi"));
try (r) { System.out.println(r.readLine()); }
// In a terminal: jshell
// jshell> 2 + 3
```

Why this helps

JShell is a REPL: read, evaluate, print, loop. The improved try syntax reduces repetition when the resource variable already exists.

Follow the execution visually



Cons, caveats and common mistakes

1. The resource must be final or effectively final.
2. The resource is closed after the try block.
3. JShell is a learning tool, not a substitute for tests.

Which approach should you choose?

Approach	Behavior	Good use case
JShell	quick experiments	exploration
try(resource)	less boilerplate	resource management

LESSON 06 | JAVA 9 | JShell and try-with-resources improvement

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can you reassign r before try(r)?

Answer and reasoning: No, then it is not effectively final.

Q2. Does try(r) close r?

Answer and reasoning: Yes, at the end of the try statement.

Q3. What is a REPL?

Answer and reasoning: An interactive environment that evaluates code as you enter it.

Practice: beginner -> intermediate -> advanced

Level 1: Start jshell and test List.of().

Level 2: Create a BufferedReader and use try(r).

Level 3: Try reassigning the resource and read compiler error.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 9 milestone: JShell and try-with-resources improvement. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | JShell and try-with-resources

Lesson 6 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can we experiment quickly and close an already-declared resource? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
BufferedReader r = new BufferedReader(reader);
try (BufferedReader x = r) { x.readLine(); }
```

AFTER: WHAT WE CAN WRITE

```
BufferedReader r = new BufferedReader(reader);
try (r) { System.out.println(r.readLine()); }
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

The variable in try(r) must be final or effectively final: assigned once. JShell is a REPL for small experiments.

INTERVIEW TRAP - PREDICT FIRST

Can you reassign r before try(r)?

ANSWER AND REASONING

No, then r is not effectively final and this form fails compilation.

TRY IT / WORKED SOLUTION

Open jshell and evaluate List.of(1,2).size(). Answer: The REPL evaluates expressions without a full source file.

LESSON 07 | JAVA 10 | var local variable inference

Real-world problem, in plain language

A long generic type is repeated on both sides of an assignment. The compiler already knows the initializer type; spelling it twice adds noise.

How developers approached it before

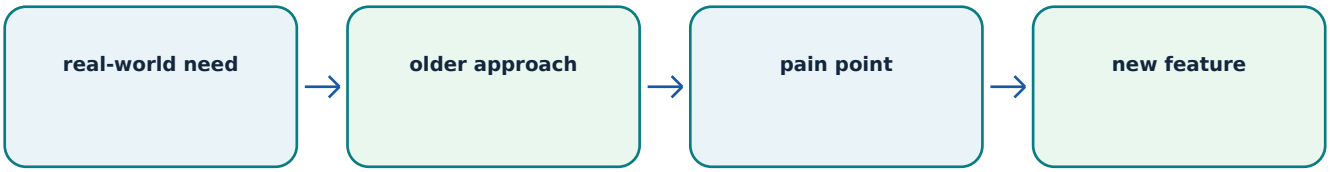
```
Map<String, List<Integer>> scores =
    new HashMap<String, List<Integer>>();
```

What is inconvenient or risky?

var asks the compiler to infer a local variable type.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 07 | JAVA 10 | var local variable inference

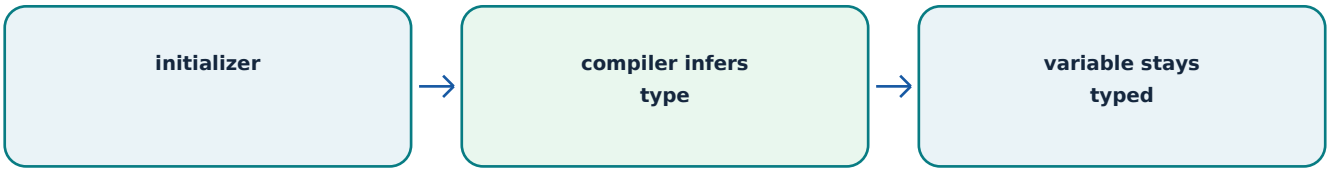
Modern approach (see version notes)

```
var scores = new HashMap<String, List<Integer>>();
var name = "Java"; // inferred String
// name = 42; // compile error
// var missing = null; // compile error
```

Why this helps

var asks the compiler to infer a local variable type. Java remains statically typed: the type is fixed at compile time.

Follow the execution visually



Cons, caveats and common mistakes

1. Only local variables and certain lambda parameters; not fields or method return types.
2. Cannot infer from null alone.
3. Avoid var when initializer hides the type.

Which approach should you choose?

Approach	Behavior	Good use case
explicit type	clear declaration	ambiguous initializer
var	less repetition	obvious initializer

LESSON 07 | JAVA 10 | var local variable inference

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does var mean dynamic typing?

Answer and reasoning: No; name remains String.

Q2. Can var be a field?

Answer and reasoning: No.

Q3. What is inferred for var x = new ArrayList<>();?

Answer and reasoning: ArrayList<Object> without more type context.

Practice: beginner -> intermediate -> advanced

Level 1: Rewrite a long generic declaration using var.

Level 2: Predict var x = 10; x = "a".

Level 3: Decide whether var improves readability in a factory call.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 10 milestone: var local variable inference. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | var inference

Lesson 7 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why repeat an obvious local variable type? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
Map<String,List<Integer>> map =
    new HashMap<String,List<Integer>>();
```

AFTER: WHAT WE CAN WRITE

```
var map = new HashMap<String,List<Integer>>();
var title = "Java";
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

var infers a compile-time type; it does not make Java dynamically typed. It works for local variables with initializers.

INTERVIEW TRAP - PREDICT FIRST

Does var x = null compile?

ANSWER AND REASONING

No. The compiler cannot infer a type from null alone.

TRY IT / WORKED SOLUTION

Try var x = 1; x = "hi";. Answer: The assignment fails: x has type int.

LESSON 08 | JAVA 10 | List.copyOf and unmodifiable collectors

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

You want a snapshot of a mutable list at the time a report is generated. A live unmodifiable view still changes when the original changes.

How developers approached it before

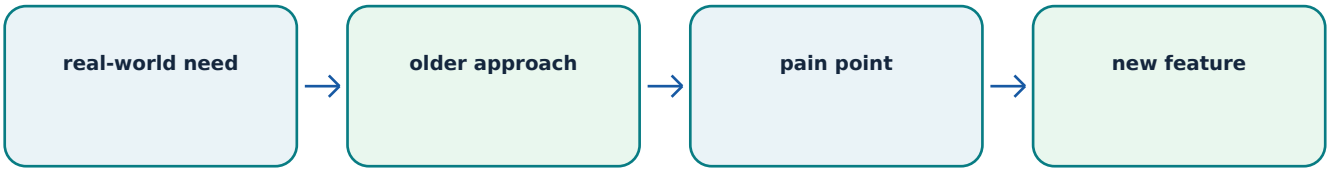
```
List<String> original = new ArrayList<>();
original.add("UPI");
List<String> view =
    Collections.unmodifiableList(original);
original.add("CARD"); // view sees CARD
```

What is inconvenient or risky?

copyOf captures element references in an unmodifiable list without keeping a live view of the mutable source.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 08 | JAVA 10 | List.copyOf and unmodifiable collectors

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

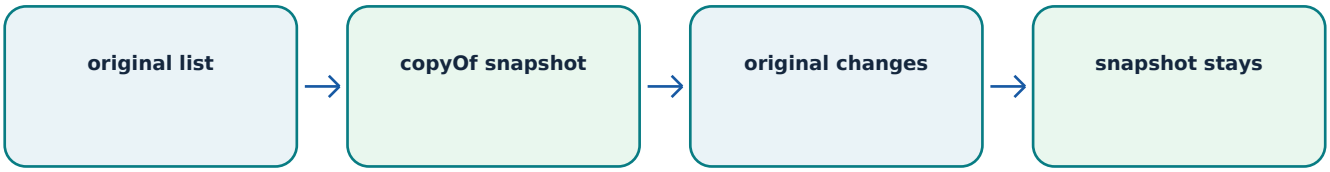
Modern approach (see version notes)

```
List<String> original = new ArrayList<>();
original.add("UPI");
List<String> snapshot = List.copyOf(original);
original.add("CARD");
System.out.println(snapshot); // [UPI]
var xs = Stream.of("a", "b")
    .collect(Collectors.toUnmodifiableList());
```

Why this helps

copyOf captures element references in an unmodifiable list without keeping a live view of the mutable source. It can reuse an already suitable immutable list.

Follow the execution visually



Cons, caveats and common mistakes

1. Shallow: mutable element objects remain shared.
2. Null elements cause NullPointerException.
3. Do not depend on reference identity of the returned list.

Which approach should you choose?

Approach	Behavior	Good use case
unmodifiableList	live view	changing state
copyOf	unmodifiable snapshot	fixed membership
new ArrayList	mutable shallow copy	editable copy

LESSON 08 | JAVA 10 | List.copyOf and unmodifiable collectors

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does copyOf deep-copy Employees?

Answer and reasoning: No; Employee objects can be shared.

Q2. What happens with a null element?

Answer and reasoning: NullPointerException.

Q3. Is copyOf guaranteed to allocate a new list?

Answer and reasoning: No; it may reuse a suitable unmodifiable list.

Practice: beginner -> intermediate -> advanced

Level 1: Compare view and snapshot after original.add().

Level 2: Try List.copyOf on a list with null.

Level 3: Use copyOf with mutable Payment and predict mutation.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 10 milestone: List.copyOf and unmodifiable collectors. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | List.copyOf and unmodifiable collectors

Reasoning, extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we give a stable read-only snapshot of a changing list? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
List<String> snap = Collections.unmodifiableList(
    new ArrayList<>(original));
```

AFTER: WHAT WE CAN WRITE

```
List<String> snap = List.copyOf(original);
var result = stream.collect(
    Collectors.toUnmodifiableList());
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

copyOf rejects null and is shallow. It may reuse an already suitable unmodifiable input.

INTERVIEW TRAP - PREDICT FIRST

If original.add("B") runs later, does snap grow?

ANSWER AND REASONING

No. The snapshot captures list membership at copy time.

TRY IT / WORKED SOLUTION

Use mutable Payment objects and mutate one after copying. Answer: The object change is visible through both lists: no deep copy occurred.

LESSON 09 | JAVA 11 | String and Files convenience APIs

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

Validating text and reading a small configuration file used to require manual trimming, loops, or encoding boilerplate.

How developers approached it before

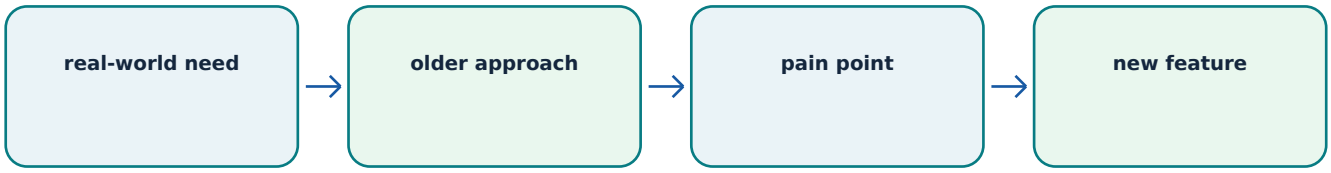
```
String s = "  hello  ";
boolean blank = s.trim().isEmpty();
byte[] bytes = Files.readAllBytes(path);
String text = new String(bytes, StandardCharsets.UTF_8);
```

What is inconvenient or risky?

strip handles Unicode whitespace more broadly than trim.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 09 | JAVA 11 | String and Files convenience APIs

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

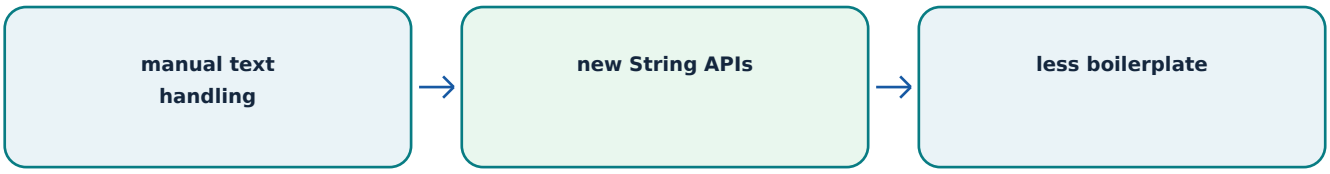
Modern approach (see version notes)

```
String s = "  hello  ";
System.out.println(s.strip());
System.out.println("  ".isBlank());
System.out.println("ab".repeat(3));
String text = Files.readString(path);
Files.writeString(path, "hello");
```

Why this helps

strip handles Unicode whitespace more broadly than trim. isBlank checks whitespace; lines splits lines; readString/writeString simplify small text-file operations.

Follow the execution visually



Cons, caveats and common mistakes

1. trim and strip are not identical.
2. Files.readString is not ideal for huge files.
3. Default readString/writeString charset is UTF-8.

Which approach should you choose?

Approach	Behavior	Good use case
trim	legacy <= U+0020 trimming	legacy code
strip	Unicode-aware whitespace	new code
readString	whole small file	configuration

LESSON 09 | JAVA 11 | String and Files convenience APIs

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Is " ".isBlank() true?

Answer and reasoning: Yes.

Q2. Does strip remove all possible visual spacing?

Answer and reasoning: No; it uses Character.isWhitespace semantics.

Q3. Should readString load a multi-gigabyte file?

Answer and reasoning: Usually no; use streaming.

Practice: beginner -> intermediate -> advanced

Level 1: Compare trim vs strip with Unicode whitespace.

Level 2: Write and read a temporary file.

Level 3: Predict "ab".repeat(0).

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 11 milestone: String and Files convenience APIs. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | String and Files convenience APIs

Lesson 5.1 · Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why is common text and file processing so verbose? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
String s = "  hi  ";
boolean blank = s.trim().isEmpty();
String text = new String(Files.readAllBytes(p), UTF_8);
```

AFTER: WHAT WE CAN WRITE

```
boolean blank = "  ".isBlank();
String text = Files.readString(p);
Files.writeString(p, "hello");
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

strip is Unicode-aware; trim uses an older whitespace rule. readString uses UTF-8 by default.

INTERVIEW TRAP - PREDICT FIRST

Is "\u2003".trim().isEmpty() always equal to isBlank()?

ANSWER AND REASONING

No. Some Unicode whitespace is recognized by isBlank but not trim.

TRY IT / WORKED SOLUTION

Try lines() on a multiline string. Answer: lines() produces a stream of lines, without line terminators.

LESSON 10 | JAVA 11 | HTTP Client

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A service calls another API. HttpURLConnection is verbose, while the newer client offers a fluent API and asynchronous calls.

How developers approached it before

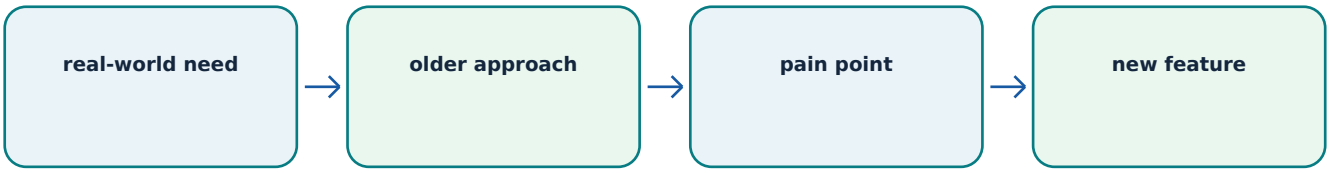
```
URL url = new URL("https://example.com");
HttpURLConnection con =
    (HttpURLConnection) url.openConnection();
con.setRequestMethod("GET");
int status = con.getResponseCode();
```

What is inconvenient or risky?

HttpClient supports HTTP/1.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 10 | JAVA 11 | HTTP Client

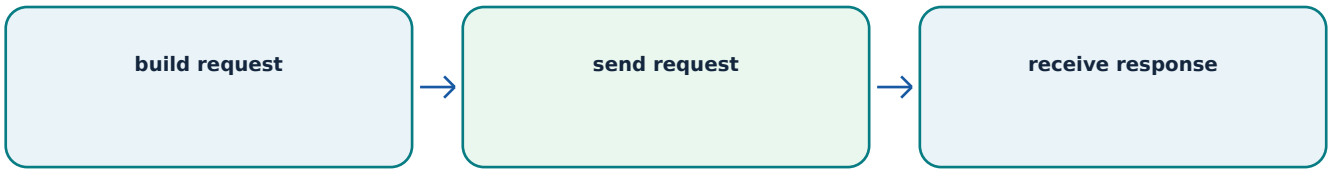
Modern approach (see version notes)

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(
    URI.create("https://example.com")).GET().build();
HttpResponse<String> res = client.send(req,
    HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());
```

Why this helps

HttpClient supports HTTP/1.1 and HTTP/2, synchronous send and asynchronous sendAsync. A request describes what to send; a response holds status and body.

Follow the execution visually



Cons, caveats and common mistakes

1. send may block and throws checked exceptions.
2. Set timeouts and handle non-2xx responses.
3. Never assume a remote service is always reachable.

Which approach should you choose?

Approach	Behavior	Good use case
HttpURLConnection	older low-level API	legacy systems
HttpClient	modern HTTP API	service integrations

LESSON 10 | JAVA 11 | HTTP Client

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does 404 automatically throw IOException?

Answer and reasoning: No; inspect statusCode().

Q2. Does sendAsync block the calling thread waiting for response?

Answer and reasoning: No; it returns CompletableFuture.

Q3. Why reuse HttpClient?

Answer and reasoning: It manages connections and configuration efficiently.

Practice: beginner -> intermediate -> advanced

Level 1: Add a request timeout.

Level 2: Handle 404 separately from 200.

Level 3: Use sendAsync and thenApply to extract body.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 11 milestone: HTTP Client. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | HTTP Client

Lesson 10 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why replace awkward HTTP connection boilerplate? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
URL u = new URL("https://example.com");
HttpURLConnection c =
    (HttpURLConnection) u.openConnection();
```

AFTER: WHAT WE CAN WRITE

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(
    URI.create("https://example.com")).build();
HttpResponse<String> r = client.send(req,
    HttpResponse.BodyHandlers.ofString());
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

HttpClient supports HTTP/2 and async APIs. Network calls can fail; handle IOException and InterruptedException.

INTERVIEW TRAP - PREDICT FIRST

Does send() block the calling thread?

ANSWER AND REASONING

Yes. sendAsync() returns a CompletableFuture instead.

TRY IT / WORKED SOLUTION

Set a timeout and inspect statusCode(). Answer: A 404 response is an HTTP response, not necessarily a Java exception.

LESSON 11 | JAVA 11 | Optional.isEmpty, Predicate.not, source launcher

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A stream needs to exclude blank names, and a command-line learner wants to run a small single-file program without a separate javac command.

How developers approached it before

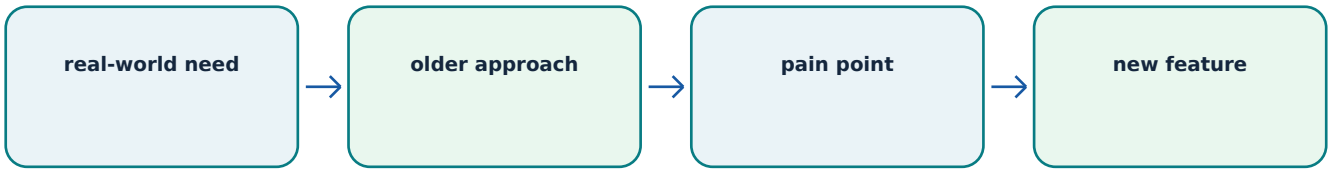
```
if (!value.isPresent()) { /* absent */ }
stream.filter(x -> !x.isBlank());
```

What is inconvenient or risky?

isEmpty expresses absence directly; Predicate.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 11 | JAVA 11 | Optional.isEmpty, Predicate.not, source launcher

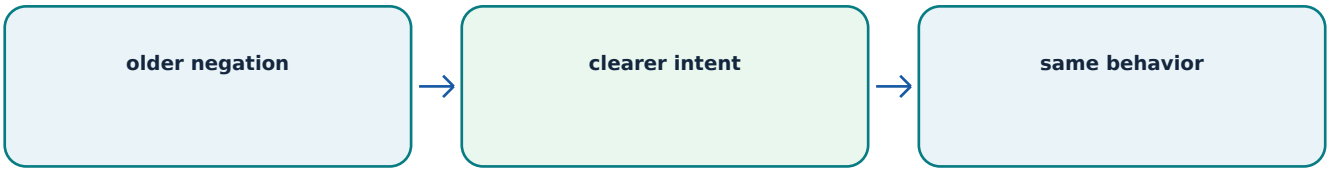
Modern approach (see version notes)

```
if (value.isEmpty()) { /* absent */ }
stream.filter(Predicate.not(String::isBlank));
// java Hello.java
// runs a source-file program directly
```

Why this helps

isEmpty expresses absence directly; Predicate.not helps negate method references; the source launcher reduces steps for small experiments.

Follow the execution visually



Cons, caveats and common mistakes

1. Predicate.not does not change semantics.
2. Source-file launch is not a replacement for project builds.
3. String.isBlank is also Java 11.

Which approach should you choose?

Approach	Behavior	Good use case
!isPresent	older expression	Java 8
isEmpty	direct absence check	Java 11+

LESSON 11 | JAVA 11 | Optional.isEmpty, Predicate.not, source launcher

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does isEmpty change Optional behavior?

Answer and reasoning: No; it is a convenience method.

Q2. What does Predicate.not(String::isBlank) keep?

Answer and reasoning: Nonblank strings.

Q3. Does source launch mean Java stops compiling?

Answer and reasoning: No; compilation is performed as part of launching.

Practice: beginner -> intermediate -> advanced

Level 1: Filter blank strings using both forms.

Level 2: Run a one-file Java program.

Level 3: Explain why isEmpty improves readability.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 11 milestone: Optional.isEmpty, Predicate.not, source launcher. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Optional.isEmpty / Predicate.not / source launcher

Lesson 02 - Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Can small APIs make intent clearer? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
if (!maybe.isPresent()) { fallback(); }
stream.filter(s -> !s.isBlank());
```

AFTER: WHAT WE CAN WRITE

```
if (maybe.isEmpty()) { fallback(); }
stream.filter(Predicate.not(String::isBlank));
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Predicate.not is useful for method references; source-file launch runs a .java file directly.

INTERVIEW TRAP - PREDICT FIRST

Does java Hello.java mean no compilation occurs?

ANSWER AND REASONING

No. The launcher compiles source in memory before running it.

TRY IT / WORKED SOLUTION

Try java Hello.java with a main method. Answer: It can launch a single source file without a separate javac command.

LESSON 12 | JAVA 12 | Switch expressions preview

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

Assigning a result from switch in Java 8 needs a temporary variable and break statements. Missing a break can accidentally fall through.

How developers approached it before

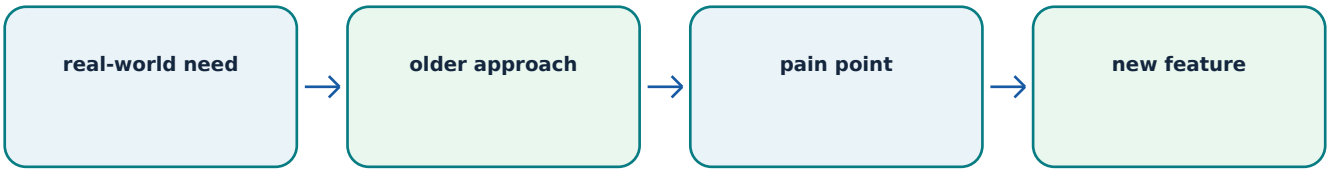
```
String kind;
switch (day) {
  case 1: kind = "work"; break;
  default: kind = "other";
}
```

What is inconvenient or risky?

Java 12 began previewing switch expressions.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 12 | JAVA 12 | Switch expressions preview

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

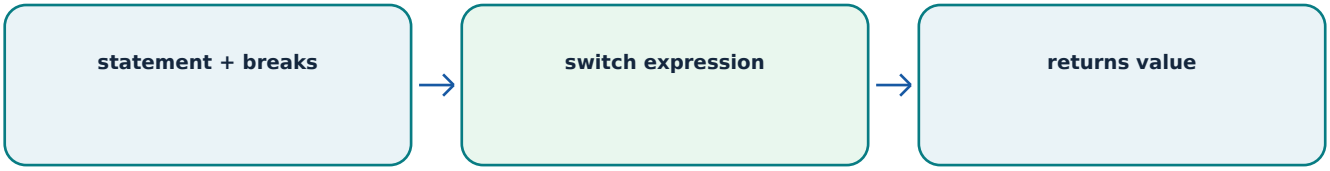
Modern approach (see version notes)

```
// Java 12: preview, syntax evolved later.
// Stable Java 14+ form:
String kind = switch (day) {
  case 1, 2, 3, 4, 5 -> "work";
  default -> "other";
};
```

Why this helps

Java 12 began previewing switch expressions. The feature evolved through previews and became final in Java 14. Learn the final syntax on JDK 21, but understand the timeline.

Follow the execution visually



Cons, caveats and common mistakes

1. Java 12 preview syntax is not identical to final syntax.
2. Do not label final arrow syntax as Java 12-final.
3. A switch expression must be exhaustive.

Which approach should you choose?

Approach	Behavior	Good use case
Java 12	preview begins	historical evolution
Java 14	final switch expressions	production syntax

LESSON 12 | JAVA 12 | Switch expressions preview

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. When was switch expression finalized?

Answer and reasoning: Java 14.

Q2. Does arrow syntax fall through?

Answer and reasoning: No.

Q3. Why is default useful?

Answer and reasoning: It handles unmatched inputs and can satisfy exhaustiveness.

Practice: beginner -> intermediate -> advanced

Level 1: Convert a Java 8 switch statement to final switch expression.

Level 2: Predict output for each day.

Level 3: Explain preview versus finalized features.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 12 milestone: Switch expressions preview. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Switch preview Java 12

Lesson 12 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why was computing a value with switch cumbersome? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
String label;
switch(day) {
 case 1: label="Mon"; break;
 default: label="Other";
}
```

AFTER: WHAT WE CAN WRITE

```
String label = switch(day) {
 case 1 -> "Mon";
 default -> "Other";
};
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Java 12 offered a preview; switch expressions became final in Java 14. Use final Java 14+ syntax in JDK 21.

INTERVIEW TRAP - PREDICT FIRST

Can a switch expression omit a matching result?

ANSWER AND REASONING

It must be exhaustive; a default or full enum coverage can satisfy this.

TRY IT / WORKED SOLUTION

Try case 1, 2 -> "Week start";. Answer: Several labels can share one result without fall-through.

LESSON 13 | JAVA 13 | Text blocks preview

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

SQL queries and JSON strings become unreadable when every line needs quotes and escaped newlines.

How developers approached it before

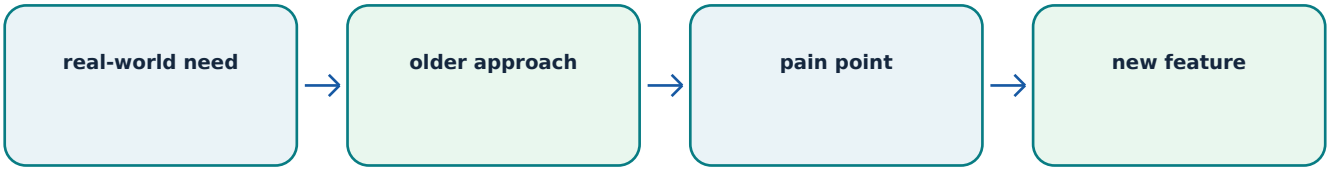
```
String json = "{
" +
    "  "name": "Ada"
" +
    "}";
```

What is inconvenient or risky?

Text blocks allow multiline string literals.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 13 | JAVA 13 | Text blocks preview

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

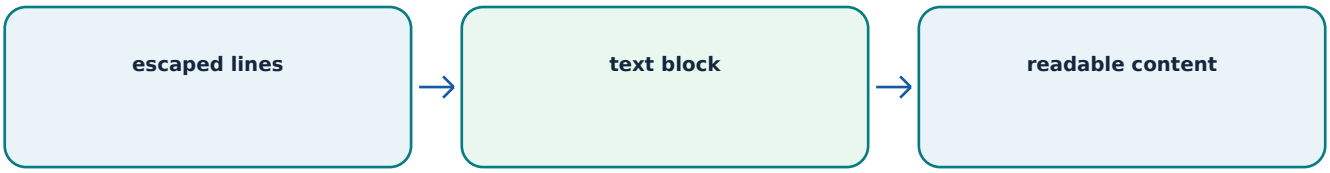
Modern approach (see version notes)

```
// Java 13 preview; final in Java 15
String json = """
    {
      "name": "Ada"
    }
    """;
```

Why this helps

Text blocks allow multiline string literals. The compiler handles incidental indentation; text blocks are still Strings.

Follow the execution visually



Cons, caveats and common mistakes

1. Whitespace and trailing newline matter.
2. Text blocks do not interpolate variables automatically.
3. The Java 13 form was preview, not final.

Which approach should you choose?

Approach	Behavior	Good use case
concatenation	manual escapes	older code
text block	multiline literal	SQL/JSON samples

LESSON 13 | JAVA 13 | Text blocks preview

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Are text blocks a new String type?

Answer and reasoning: No; the result is java.lang.String.

Q2. When finalized?

Answer and reasoning: Java 15.

Q3. Do they automatically substitute ${name}?

Answer and reasoning: No.

Practice: beginner -> intermediate -> advanced

Level 1: Convert multiline SQL concatenation to a text block.

Level 2: Compare trailing newline behavior.

Level 3: Use formatted() to insert values safely where appropriate.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 13 milestone: Text blocks preview. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Text blocks preview Java 13

Lesson 13 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can we write readable multiline JSON or SQL? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
String json = "{\n" +
    "  \"name\": \"Ada\"\n" + "}";
```

AFTER: WHAT WE CAN WRITE

```
String json = """
    {
      "name": "Ada"
    }
    """;
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Text blocks previewed in Java 13 and became final in Java 15. Incidental indentation and final newline matter.

INTERVIEW TRAP - PREDICT FIRST

Does a text block automatically create valid JSON?

ANSWER AND REASONING

No. It only creates a String; content still must be valid JSON.

TRY IT / WORKED SOLUTION

Move the closing delimiter and inspect whitespace. Answer: Closing delimiter position affects indentation removal.

LESSON 14 | JAVA 14 | Final switch expressions

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A discount category must always map to one price. Java 8 switch statements require assignments and can accidentally omit cases.

How developers approached it before

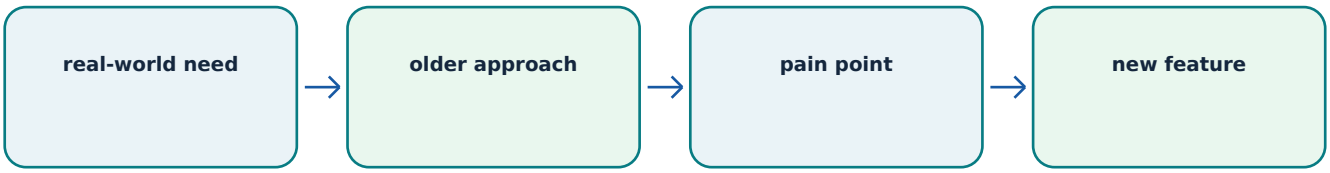
```
int fee;
switch (tier) {
 case "GOLD": fee = 0; break;
 case "SILVER": fee = 5; break;
 default: fee = 10;
}
```

What is inconvenient or risky?

The final feature makes a switch return a value.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 14 | JAVA 14 | Final switch expressions

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

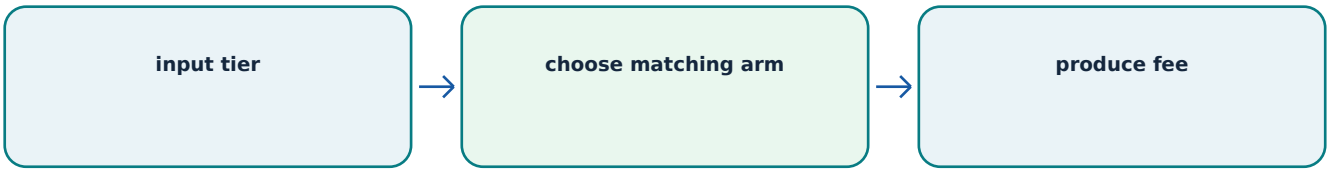
Modern approach (see version notes)

```
int fee = switch (tier) {
  case "GOLD" -> 0;
  case "SILVER" -> 5;
  default -> 10;
};
```

Why this helps

The final feature makes a switch return a value. Arrow cases do not fall through, and the compiler checks exhaustiveness.

Follow the execution visually



Cons, caveats and common mistakes

1. Use yield inside a block arm.
2. Switch expression and switch statement are different constructs.
3. Null input may throw unless explicitly supported by later pattern switch.

Which approach should you choose?

Approach	Behavior	Good use case
switch statement	control flow	side effects
switch expression	produces value	mapping input

LESSON 14 | JAVA 14 | Final switch expressions

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does arrow case fall through?

Answer and reasoning: No.

Q2. What keyword returns from a block arm?

Answer and reasoning: yield.

Q3. What if no arm covers a possible value?

Answer and reasoning: The switch expression must be exhaustive.

Practice: beginner -> intermediate -> advanced

Level 1: Rewrite a fee calculator.

Level 2: Use a block arm with yield.

Level 3: Compare arrow syntax with colon statement syntax.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 14 milestone: Final switch expressions. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Switch expressions final Java 14

Lesson 14 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can a switch produce a value without accidental fall-through? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
int fee;
switch(type) {
 case "UPI": fee=0; break;
 default: fee=10;
}
```

AFTER: WHAT WE CAN WRITE

```
int fee = switch(type) {
 case "UPI" -> 0;
 default -> 10;
};
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Arrow labels do not fall through. A block arm uses yield to return its result.

INTERVIEW TRAP - PREDICT FIRST

Can you use break 10; to return a switch-expression value?

ANSWER AND REASONING

No. Use yield 10; in a block arm.

TRY IT / WORKED SOLUTION

Add case "CARD" -> { yield 5; }. Answer: A block can compute a value before yielding.

LESSON 15 | JAVA 14 | Helpful NullPointerExceptions

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A chained expression fails with NullPointerException, but older error messages often make it hard to identify which part was null.

How developers approached it before

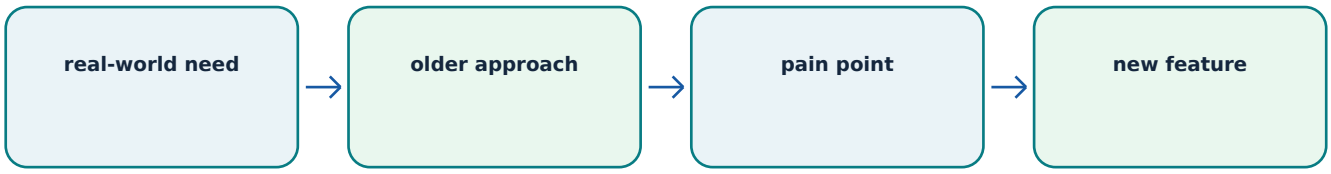
```
String city = user.getAddress().getCity();
// Old NPE message may not identify which
// expression was null.
```

What is inconvenient or risky?

Helpful NPE messages explain which dereference failed.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 15 | JAVA 14 | Helpful NullPointerExceptions

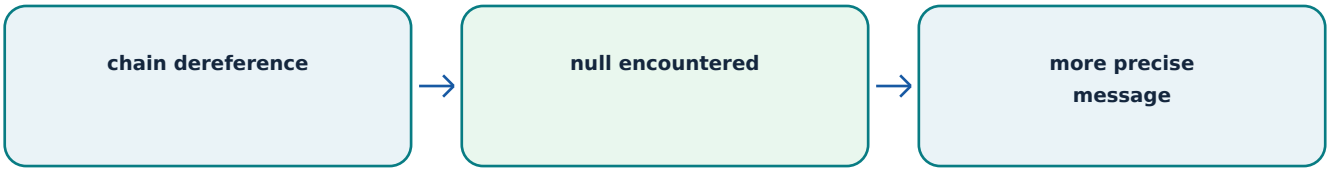
Modern approach (see version notes)

```
String city = user.getAddress().getCity();
// Helpful NPE can identify that getAddress()
// returned null, depending on runtime settings.
```

Why this helps

Helpful NPE messages explain which dereference failed. They improve debugging but do not remove null problems.

Follow the execution visually



Cons, caveats and common mistakes

1. Do not parse NPE messages as stable program logic.
2. The feature does not prevent NPEs.
3. Check nullable values or model absence explicitly.

Which approach should you choose?

Approach	Behavior	Good use case
old NPE	less diagnostic detail	legacy
helpful NPE	identifies failing dereference	debugging

LESSON 15 | JAVA 14 | Helpful NullPointerExceptions

Frequently asked and tricky interview questions - with reasoning

Q1. Does helpful NPE prevent the exception?

Answer and reasoning: No.

Q2. Can messages vary by context?

Answer and reasoning: Yes.

Q3. Should business logic depend on message text?

Answer and reasoning: No.

Practice: beginner -> intermediate -> advanced

Level 1: Create a two-level null chain and inspect error.

Level 2: Refactor with null checks.

Level 3: Compare Optional with nullable field design.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 14 milestone: Helpful NullPointerExceptions. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Helpful NullPointerExceptions

Lesson 15 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can a stack trace identify which link in a chain was null? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
String city = user.getAddress().getCity();
// NPE gives less precise context
```

AFTER: WHAT WE CAN WRITE

```
String city = user.getAddress().getCity();
// modern JVM often identifies the null link
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Helpful NPE messages improve debugging, but they do not prevent nulls. Exact text can vary.

INTERVIEW TRAP - PREDICT FIRST

Does this replace validation?

ANSWER AND REASONING

No. It improves diagnostics after failure; validation still matters.

TRY IT / WORKED SOLUTION

Make getAddress() return null and inspect error. Answer: The message points toward the null-returning expression.

LESSON 16 | JAVA 15 | Final text blocks

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A team embeds example JSON in a test. Concatenated string literals obscure the actual document.

How developers approached it before

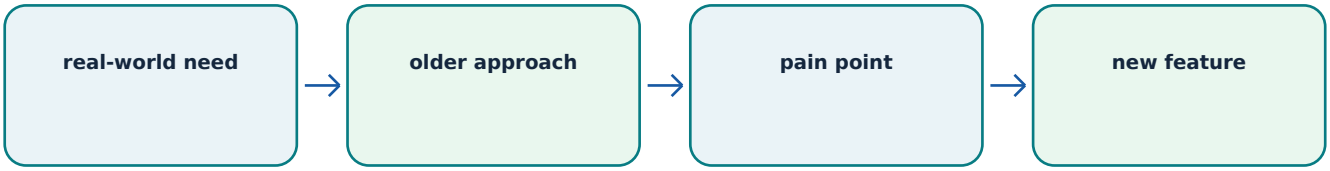
```
String sql = "SELECT id, name
" +
    "FROM users
" +
    "WHERE active = true";
```

What is inconvenient or risky?

Java 15 finalized text blocks after preview rounds.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 16 | JAVA 15 | Final text blocks

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

Modern approach (see version notes)

```
String sql = """
    SELECT id, name
    FROM users
    WHERE active = true
    """;
```

Why this helps

Java 15 finalized text blocks after preview rounds. They reduce quoting noise and keep multiline text visually close to its intended form.

Follow the execution visually



Cons, caveats and common mistakes

1. Trailing newline may differ from old string.
2. Indentation rules affect resulting whitespace.
3. SQL parameters should still use prepared statements.

Which approach should you choose?

Approach	Behavior	Good use case
concatenation	manual line breaks	old code
text block	natural multiline	SQL/JSON/tests

LESSON 16 | JAVA 15 | Final text blocks

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Do text blocks protect against SQL injection?

Answer and reasoning: No.

Q2. Is a text block a String?

Answer and reasoning: Yes.

Q3. Why might tests fail after conversion?

Answer and reasoning: Whitespace/newline differences.

Practice: beginner -> intermediate -> advanced

Level 1: Convert JSON and compare exact equality.

Level 2: Use stripIndent and formatted.

Level 3: Find trailing newline in output.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 15 milestone: Final text blocks. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Text blocks final Java 15

Lesson 16 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we keep embedded SQL readable? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
String sql = "SELECT id, name\n" +
    "FROM users\nWHERE active = true";
```

AFTER: WHAT WE CAN WRITE

```
String sql = """
    SELECT id, name
    FROM users
    WHERE active = true
    """;
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Text blocks are still Strings. Parameterize user input in SQL; do not concatenate unsafe values.

INTERVIEW TRAP - PREDICT FIRST

Does a text block protect against SQL injection?

ANSWER AND REASONING

No. Use PreparedStatement placeholders.

TRY IT / WORKED SOLUTION

Print sql.length() and escaped representation. Answer: Trailing newline and indentation affect the actual String.

LESSON 17 | JAVA 15 | Hidden classes and JVM context

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A framework generates temporary implementation classes at runtime. Ordinary classes have names and visibility expectations that do not fit every generated implementation.

How developers approached it before

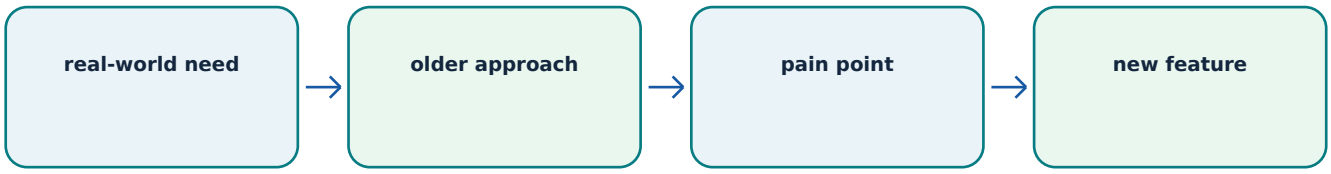
```
// Conceptual older approach:
// framework defines named helper classes
// and manages their lifetime and lookup.
```

What is inconvenient or risky?

Hidden classes are mainly a runtime/framework capability.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 17 | JAVA 15 | Hidden classes and JVM context

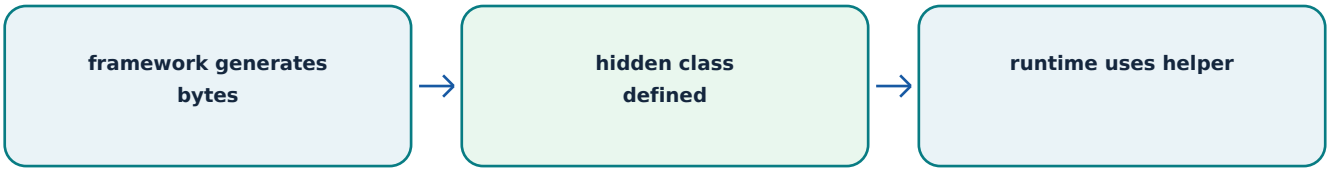
Modern approach (see version notes)

```
// Advanced API: MethodHandles.Lookup
// can define a hidden class from bytecode.
// Not a normal application-level class syntax.
```

Why this helps

Hidden classes are mainly a runtime/framework capability. Beginners should know why the JVM needs them, not memorize low-level bytecode APIs.

Follow the execution visually



Cons, caveats and common mistakes

1. Not a general replacement for normal classes.
2. Requires advanced bytecode and MethodHandles knowledge.
3. Framework developers benefit more than ordinary application code.

Which approach should you choose?

Approach	Behavior	Good use case
ordinary class	named application type	domain logic
hidden class	runtime-generated helper	framework internals

LESSON 17 | JAVA 15 | Hidden classes and JVM context

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can you declare a hidden class with a new keyword?

Answer and reasoning: No; it is defined dynamically via runtime APIs.

Q2. Is it a daily backend feature?

Answer and reasoning: Usually not.

Q3. Why does it exist?

Answer and reasoning: To support runtime-generated implementation classes.

Practice: beginner -> intermediate -> advanced

Level 1: Identify libraries that generate classes.

Level 2: Explain named vs hidden runtime helpers.

Level 3: Describe when to defer this advanced topic.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 15 milestone: Hidden classes and JVM context. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Hidden classes

Lesson 17 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why do frameworks need generated classes that users should not import? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Framework generates a named helper class
// that can be discovered by ordinary name.
```

AFTER: WHAT WE CAN WRITE

```
// Java 15 adds Lookup.defineHiddenClass(...)
// for framework-generated implementation classes.
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Advanced JVM feature, not a replacement for ordinary application classes. Typically used by runtimes and frameworks.

INTERVIEW TRAP - PREDICT FIRST

Should a business DTO be a hidden class?

ANSWER AND REASONING

No. Hidden classes serve dynamic runtime implementation needs.

TRY IT / WORKED SOLUTION

Find a framework use case: proxy or lambda support. Answer: The benefit is controlled lifecycle and non-discoverability by ordinary class name.

LESSON 18 | JAVA 16 | Records

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A DTO carries customer data but requires constructor, getters, equals, hashCode and toString. Repeating boilerplate hides the actual fields.

How developers approached it before

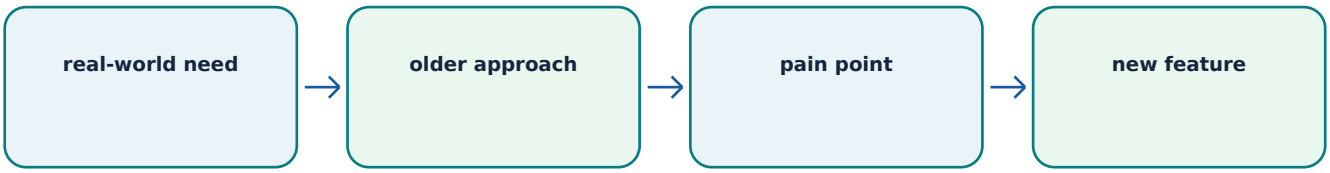
```
final class Customer {
  private final String name;
  Customer(String name) { this.name = name; }
  String getName() { return name; }
  // equals/hashCode/toString omitted
}
```

What is inconvenient or risky?

Records are concise data carriers.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 18 | JAVA 16 | Records

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

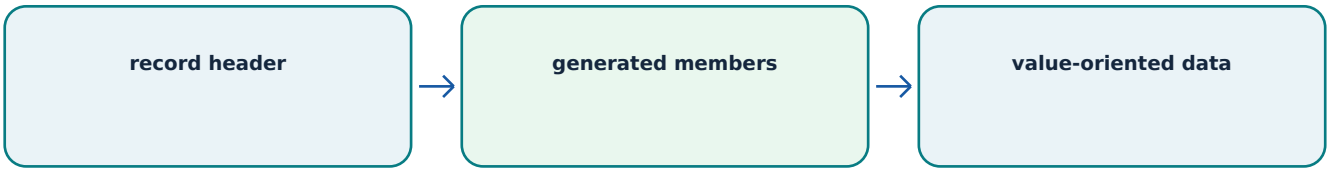
Modern approach (see version notes)

```
record Customer(String name, int age) {}
Customer c = new Customer("Ada", 30);
System.out.println(c.name());
System.out.println(c);
```

Why this helps

Records are concise data carriers. They generate a canonical constructor, accessors, equals, hashCode and toString. Component fields are final, but referenced objects may be mutable.

Follow the execution visually



Cons, caveats and common mistakes

1. Records cannot extend arbitrary classes.
2. Record components are shallowly final, not deeply immutable.
3. Validate invariants in a compact constructor.

Which approach should you choose?

Approach	Behavior	Good use case
class	flexible behavior/state	general object
record	data carrier	DTO/value-like data

LESSON 18 | JAVA 16 | Records

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can a record contain List<String>?

Answer and reasoning: Yes, but the list may still be mutable.

Q2. Does a record create getName()?

Answer and reasoning: No; accessor is name().

Q3. Can a record extend a normal class?

Answer and reasoning: No; records implicitly extend java.lang.Record.

Practice: beginner -> intermediate -> advanced

Level 1: Replace a DTO with record.

Level 2: Write a compact constructor validating age >= 0.

Level 3: Demonstrate mutable list inside a record.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 16 milestone: Records. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Records

Lesson 18 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why write constructors, getters, equals, and hashCode for data? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
final class Point {
 final int x, y;
 Point(int x,int y){this.x=x;this.y=y;}
}
```

AFTER: WHAT WE CAN WRITE

```
record Point(int x, int y) {}
Point p = new Point(2, 3);
System.out.println(p.x());
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Records are shallowly immutable carriers: component fields final, referenced objects may be mutable.

INTERVIEW TRAP - PREDICT FIRST

Does record Box(List<String> xs) freeze xs?

ANSWER AND REASONING

No. The list can still be mutable; use defensive copying when needed.

TRY IT / WORKED SOLUTION

Add a compact constructor validating x >= 0. Answer: Record constructors can enforce invariants.

LESSON 19 | JAVA 16 | Pattern matching instanceof

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A method checks an object type, then casts it. Repeating the type is noisy and a wrong cast is possible if logic changes.

How developers approached it before

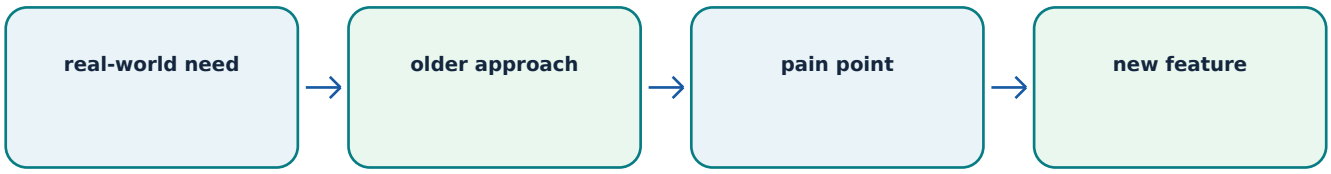
```
if (value instanceof String) {
  String text = (String) value;
  System.out.println(text.length());
}
```

What is inconvenient or risky?

Pattern matching combines the type test with a safely scoped variable.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 19 | JAVA 16 | Pattern matching instanceof

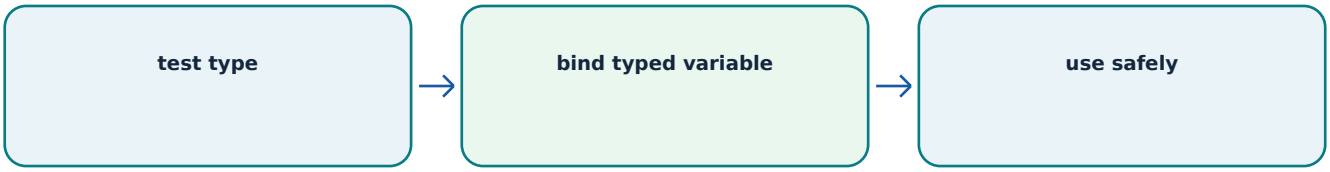
Modern approach (see version notes)

```
if (value instanceof String text) {
  System.out.println(text.length());
}
```

Why this helps

Pattern matching combines the type test with a safely scoped variable. The variable exists only where the compiler knows the match succeeded.

Follow the execution visually



Cons, caveats and common mistakes

1. Scope follows control flow; variable may not exist after the if.
2. Null does not match instanceof.
3. Avoid complicated boolean conditions that obscure scope.

Which approach should you choose?

Approach	Behavior	Good use case
test + cast	two steps	Java 8
pattern instanceof	single step	Java 16+

LESSON 19 | JAVA 16 | Pattern matching instanceof

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does null instanceof String match?

Answer and reasoning: No.

Q2. Can text be used in the else branch?

Answer and reasoning: No; match failed there.

Q3. Does this cast at runtime?

Answer and reasoning: The runtime type test ensures the binding is safe.

Practice: beginner -> intermediate -> advanced

Level 1: Rewrite instanceof + cast.

Level 2: Try accessing binding in else and inspect compiler error.

Level 3: Use with a sealed hierarchy later.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 16 milestone: Pattern matching instanceof. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Pattern matching instanceof

Lesson 19 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why check type and then repeat a cast? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
if (obj instanceof String) {
 String s = (String) obj;
 System.out.println(s.length());
}
```

AFTER: WHAT WE CAN WRITE

```
if (obj instanceof String s) {
 System.out.println(s.length());
}
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Pattern variables are available only where the compiler knows the type test succeeded.

INTERVIEW TRAP - PREDICT FIRST

Can s be used after the if block?

ANSWER AND REASONING

Not generally; its scope depends on flow analysis.

TRY IT / WORKED SOLUTION

Try if (!(obj instanceof String s)) return;. Answer: After that guard, s is in scope in the remaining path.

LESSON 20 | JAVA 16 | Stream.toList

Real-world problem, in plain language

A stream produces filtered names. Java 8 often uses Collectors.toList(), but the returned list mutability is not guaranteed by that collector contract.

How developers approached it before

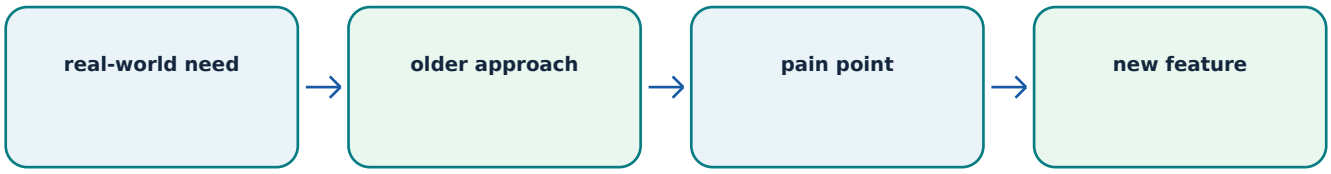
```
List<String> out = names.stream()
    .filter(x -> x.length() > 3)
    .collect(Collectors.toList());
```

What is inconvenient or risky?

Stream.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 20 | JAVA 16 | Stream.toList

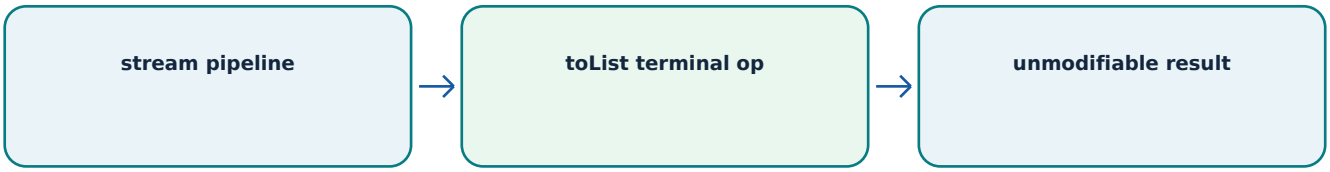
Modern approach (see version notes)

```
List<String> out = names.stream()
    .filter(x -> x.length() > 3)
    .toList();
// out.add("x"); // throws
```

Why this helps

Stream.toList returns an unmodifiable list and permits null stream elements. It is not interchangeable with every collector.

Follow the execution visually



Cons, caveats and common mistakes

1. Stream.toList is Java 16, not Java 9.
2. Collectors.toList does not promise a specific mutability.
3. Collectors.toUnmodifiableList rejects nulls.

Which approach should you choose?

Approach	Behavior	Good use case
toList()	unmodifiable; null allowed	read-only result
Collectors.toList	unspecified mutability	legacy collector
toUnmodifiableList	null rejected	strict immutable result

LESSON 20 | JAVA 16 | Stream.toList

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can you add to Stream.toList result?

Answer and reasoning: No.

Q2. Does Stream.toList allow null elements?

Answer and reasoning: Yes.

Q3. Does Collectors.toList guarantee ArrayList?

Answer and reasoning: No; avoid relying on implementation.

Practice: beginner -> intermediate -> advanced

Level 1: Compare all three list collectors.

Level 2: Stream.of("a", null).toList().

Level 3: Make a mutable copy with new ArrayList<>(result).

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 16 milestone: Stream.toList. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Stream.toList

Lesson 20 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we collect a stream with less boilerplate? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
List<String> x = stream.collect(
    Collectors.toList());
```

AFTER: WHAT WE CAN WRITE

```
List<String> x = stream.toList();
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Stream.toList() returns an unmodifiable list; Collectors.toList() makes no guarantee of list type or mutability.

INTERVIEW TRAP - PREDICT FIRST

Does stream.toList().add("X") work?

ANSWER AND REASONING

No, it throws UnsupportedOperationException.

TRY IT / WORKED SOLUTION

Compare Stream.of("A", null).toList() with List.of. Answer: Stream.toList can contain null; List.of rejects null.

LESSON 21 | JAVA 17 | Sealed classes

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A payment result can be Success or Failure, but unrelated classes should not implement the result interface. An open hierarchy allows unexpected implementations.

How developers approached it before

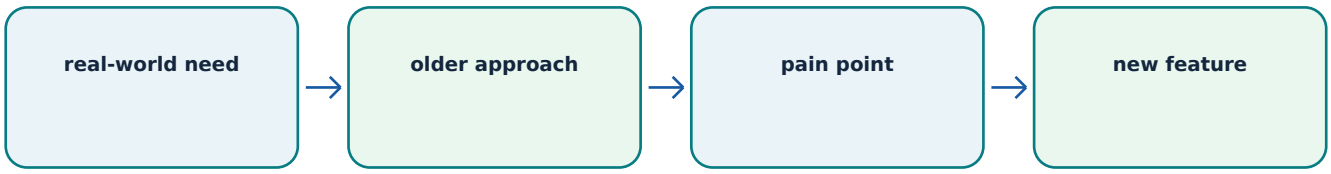
```
interface Result {}
class Success implements Result {}
class Failure implements Result {}
// Any other class can implement Result.
```

What is inconvenient or risky?

Sealed types restrict the permitted direct subclasses.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 21 | JAVA 17 | Sealed classes

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

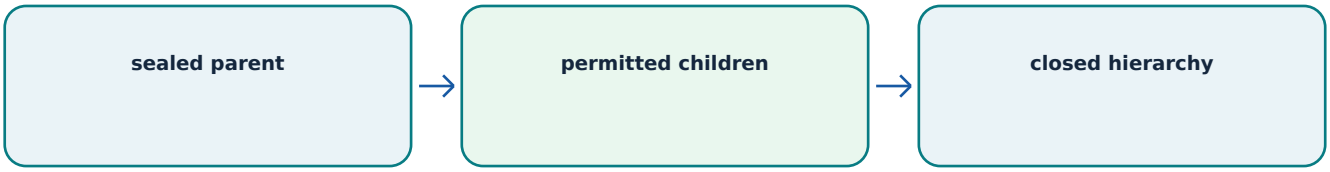
Modern approach (see version notes)

```
sealed interface Result permits Success, Failure {}
record Success(String id) implements Result {}
record Failure(String reason) implements Result {}
```

Why this helps

Sealed types restrict the permitted direct subclasses. This makes the model easier to reason about and supports exhaustive pattern switches.

Follow the execution visually



Cons, caveats and common mistakes

1. Permitted direct subclasses must be final, sealed or non-sealed (records are final).
2. Sealed is not the same as immutable.
3. Use open interfaces when extensibility is intended.

Which approach should you choose?

Approach	Behavior	Good use case
open interface	any implementation	plugin extension
sealed interface	restricted implementations	domain alternatives

LESSON 21 | JAVA 17 | Sealed classes

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can an arbitrary class implement Result?

Answer and reasoning: No; it must be permitted.

Q2. Does sealed freeze object fields?

Answer and reasoning: No.

Q3. Why pair sealed types with switch?

Answer and reasoning: The compiler can reason about all permitted cases.

Practice: beginner -> intermediate -> advanced

Level 1: Model Success/Failure using sealed interface.

Level 2: Add a third permitted case.

Level 3: Explain final vs sealed vs non-sealed.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 17 milestone: Sealed classes. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Sealed classes

Lesson 21 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can a domain limit which types are allowed? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
interface Payment {}
class Card implements Payment {}
class Cash implements Payment {}
```

AFTER: WHAT WE CAN WRITE

```
sealed interface Payment permits Card, Cash {}
final class Card implements Payment {}
final class Cash implements Payment {}
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Permitted subclasses must declare final, sealed, or non-sealed (with applicable rules).

INTERVIEW TRAP - PREDICT FIRST

Can an arbitrary third class implement Payment?

ANSWER AND REASONING

No, unless the sealed hierarchy permits it.

TRY IT / WORKED SOLUTION

Make Cash non-sealed and add a subclass. Answer: A non-sealed permitted class opens that branch to extension.

LESSON 22 | JAVA 17 | Strong encapsulation and migration

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

An old dependency uses internal JDK classes or reflective access to private fields. A program that worked on Java 8 may fail after upgrading.

How developers approached it before

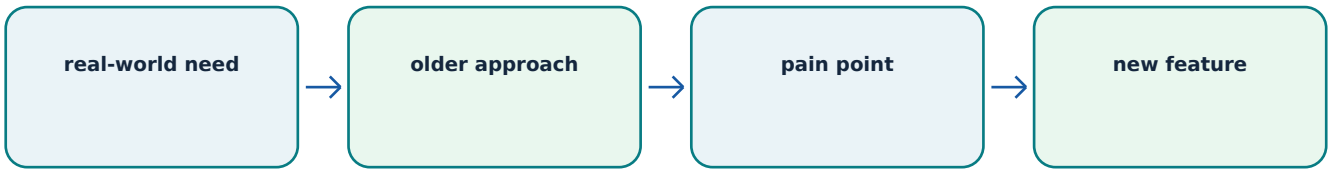
```
// Fragile old library assumption:
// reflect into non-public JDK internals
// and assume it always works.
```

What is inconvenient or risky?

Java 17 strongly encapsulates JDK internals.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 22 | JAVA 17 | Strong encapsulation and migration

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

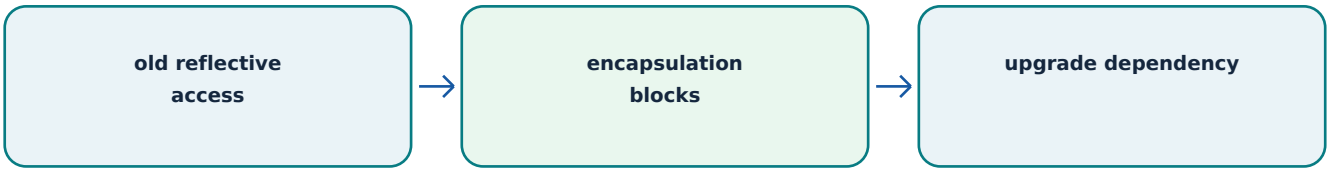
Modern approach (see version notes)

```
// Prefer supported public APIs.
// Inspect migration warnings and update
// dependencies before removing workarounds.
// java --illegal-access is not a Java 21 fix.
```

Why this helps

Java 17 strongly encapsulates JDK internals. The right migration strategy is to upgrade libraries and remove unsupported access, not rely indefinitely on access-opening flags.

Follow the execution visually



Cons, caveats and common mistakes

1. --add-opens can be a targeted temporary workaround.
2. javax.* APIs removed from JDK 11 may need dependencies.
3. Compile/test with actual target JDK.

Which approach should you choose?

Approach	Behavior	Good use case
supported API	stable contract	preferred
JDK internals	implementation detail	migration risk

LESSON 22 | JAVA 17 | Strong encapsulation and migration

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Is --add-opens a permanent design solution?

Answer and reasoning: Usually no; prefer supported APIs.

Q2. Does classpath avoid all encapsulation?

Answer and reasoning: No.

Q3. What should be upgraded first?

Answer and reasoning: Dependencies and build tools that rely on JDK internals.

Practice: beginner -> intermediate -> advanced

Level 1: List dependencies and inspect Java 21 compatibility.

Level 2: Explain exports vs opens.

Level 3: Run jdeps on a legacy application.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 17 milestone: Strong encapsulation and migration. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Strong encapsulation

Lesson 22 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why can older reflection-heavy libraries break after a JDK upgrade? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Older libraries may reflect into
// private JDK implementation fields.
```

AFTER: WHAT WE CAN WRITE

```
// Prefer public APIs; upgrade libraries.
// --add-opens is a targeted temporary bridge.
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Java 17 strongly encapsulates most JDK internals. Module opens is not the same as exports.

INTERVIEW TRAP - PREDICT FIRST

Does --add-opens make a package a normal public API?

ANSWER AND REASONING

No. It permits deep reflection to the named target, not general compile-time imports.

TRY IT / WORKED SOLUTION

Run an older dependency with --illegal-access assumptions. Answer: Identify the failing reflective access and update the library.

LESSON 23 | JAVA 18 | UTF-8 default charset

Real-world problem, in plain language

A program writes text without specifying encoding. On older machines, the default charset could differ across operating systems, producing inconsistent files.

How developers approached it before

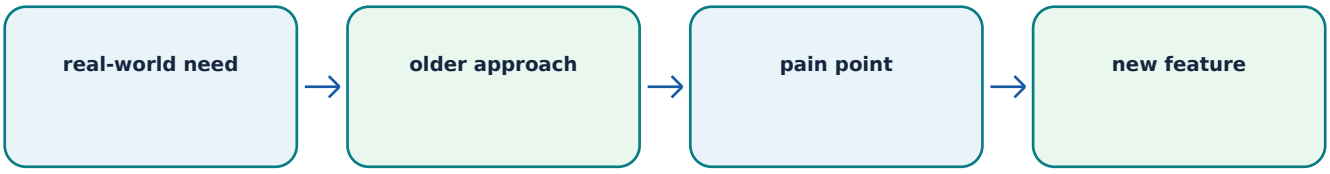
```
Files.write(path, "cafe".getBytes());
// Earlier default charset may vary by machine.
```

What is inconvenient or risky?

A charset maps characters to bytes.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 23 | JAVA 18 | UTF-8 default charset

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

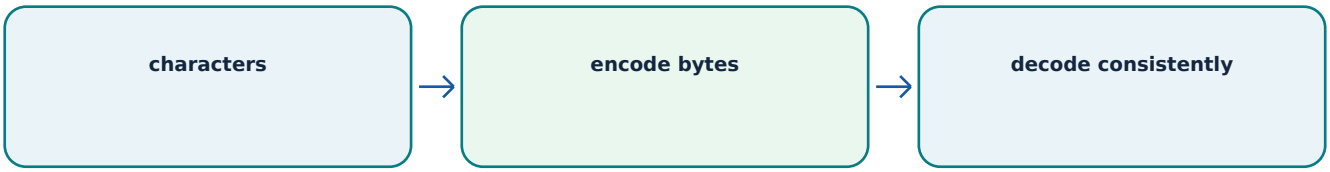
Modern approach (see version notes)

```
// Java 18 default charset is UTF-8 (JEP 400).
Files.writeString(path, "cafe");
// For portable intent, be explicit:
Files.writeString(path, "cafe",
    StandardCharsets.UTF_8);
```

Why this helps

A charset maps characters to bytes. Java 18 standardized the default to UTF-8 for most standard APIs, reducing cross-platform surprises.

Follow the execution visually



Cons, caveats and common mistakes

1. Some APIs/configurations have exceptions.
2. Explicit charset remains clearest for protocols/files.
3. Byte-level compatibility matters for legacy data.

Which approach should you choose?

Approach	Behavior	Good use case
platform default	machine-dependent historically	legacy
UTF-8	consistent default	modern portability

LESSON 23 | JAVA 18 | UTF-8 default charset

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. What is a charset?

Answer and reasoning: A rule for encoding characters as bytes and decoding bytes as text.

Q2. Why might old files break?

Answer and reasoning: They may have been encoded with a different default charset.

Q3. Should network protocols omit charset?

Answer and reasoning: Prefer explicit protocol-specified encoding.

Practice: beginner -> intermediate -> advanced

Level 1: Write non-ASCII text with explicit UTF-8.

Level 2: Inspect bytes in hex.

Level 3: Explain why text can become garbled.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 18 milestone: UTF-8 default charset. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | UTF-8 default charset

Lesson 23 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why can text behave differently on two machines? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
byte[] b = text.getBytes();
// Old default charset varied by environment
```

AFTER: WHAT WE CAN WRITE

```
byte[] b = text.getBytes();
// Java 18 default charset is UTF-8
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Always pass StandardCharsets.UTF_8 when an explicit file or protocol encoding matters.

INTERVIEW TRAP - PREDICT FIRST

Does Java 18 make every file UTF-8?

ANSWER AND REASONING

No. Existing files and external protocols can use other encodings.

TRY IT / WORKED SOLUTION

Read a known non-UTF-8 file explicitly. Answer: Use the correct Charset; do not rely on platform defaults.

LESSON 24 | JAVA 18 | Simple Web Server and Javadoc snippets

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

You want to share static HTML files locally and embed maintainable code snippets in API documentation without running a full application server.

How developers approached it before

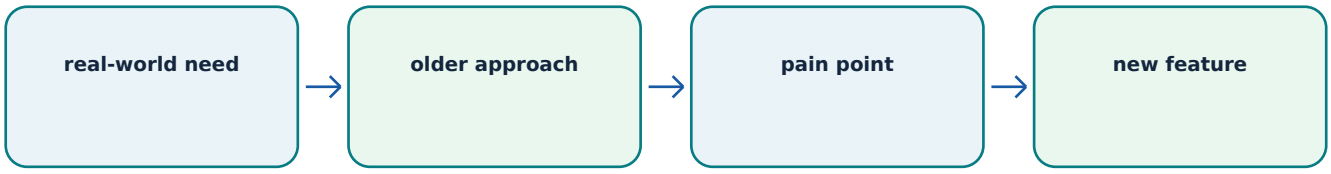
```
// Before: install a separate local server
// or build custom server code for static files.
```

What is inconvenient or risky?

jwebserver serves static files for local testing; @snippet improves code examples in Javadoc.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 24 | JAVA 18 | Simple Web Server and Javadoc snippets

Modern approach (see version notes)

```
// JDK 18+ tool (development use):
// jwebserver -p 8000
// Javadoc supports @snippet for code examples.
```

Why this helps

jwebserver serves static files for local testing; @snippet improves code examples in Javadoc. Neither replaces production application infrastructure.

Follow the execution visually



Cons, caveats and common mistakes

1. Do not treat jwebserver as production-hardened.
2. Snippet content must be maintained and tested.
3. jwebserver does not run arbitrary backend endpoints.

Which approach should you choose?

Approach	Behavior	Good use case
jwebserver	static local files	quick demo
@snippet	Javadoc code examples	documentation

LESSON 24 | JAVA 18 | Simple Web Server and Javadoc snippets

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Is jwebserver a full application server?

Answer and reasoning: No; it is a simple static-file server.

Q2. Why use @snippet?

Answer and reasoning: To present clearer, maintainable code examples in docs.

Q3. Should it face the public internet?

Answer and reasoning: Not as a production server.

Practice: beginner -> intermediate -> advanced

Level 1: Serve a small index.html locally.

Level 2: Add a Javadoc snippet to a method.

Level 3: Compare static content and API endpoint.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 18 milestone: Simple Web Server and Javadoc snippets. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Simple Web Server and Javadoc snippets

Lesson 24 Extra: examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can we serve static files quickly and document code accurately? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Need a separate static file server
// or custom HTTP server setup.
```

AFTER: WHAT WE CAN WRITE

```
// Terminal: jwebserver -p 8000
// Javadoc: @snippet :
//   int answer = 42;
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

jwebserver is for development/testing, not a production app server. Snippets help keep examples structured.

INTERVIEW TRAP - PREDICT FIRST

Does jwebserver replace Spring Boot?

ANSWER AND REASONING

No. It is a minimal static file server.

TRY IT / WORKED SOLUTION

Serve a directory and fetch a static HTML file. Answer: It is convenient for local demos and testing.

LESSON 25 | JAVA 19 | Virtual threads first preview

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A web service handles many slow I/O requests. Traditional platform threads are relatively expensive when thousands spend time waiting. Java 19 previewed virtual threads.

How developers approached it before

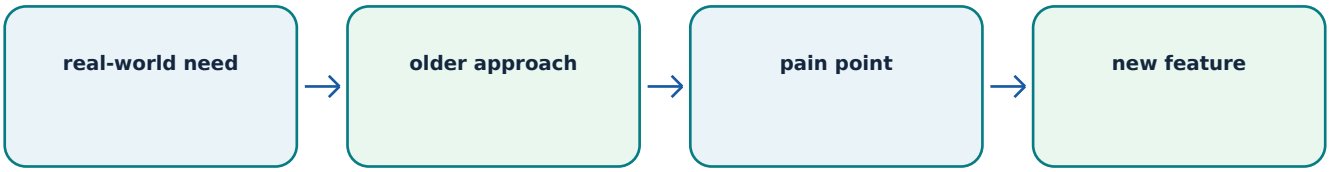
```
ExecutorService pool =
    Executors.newFixedThreadPool(100);
// Each blocking task occupies a pool thread.
```

What is inconvenient or risky?

Virtual threads are lightweight Java threads that can be parked while waiting for many blocking operations.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 25 | JAVA 19 | Virtual threads first preview

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

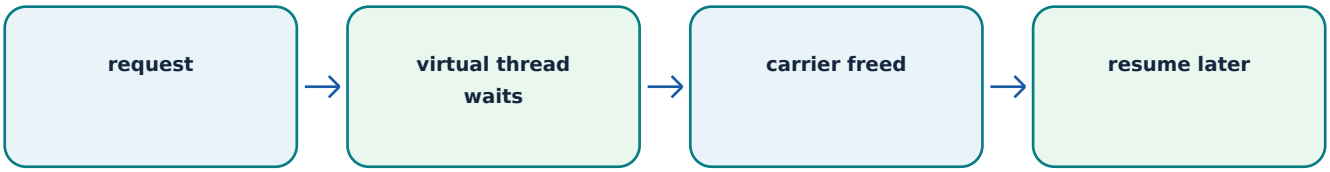
Modern approach (see version notes)

```
// Java 19: preview API; learn final Java 21 API:
try (var executor =
    Executors.newVirtualThreadPerTaskExecutor()) {
  executor.submit(() -> fetchRemoteData()); // helper assumed
}
```

Why this helps

Virtual threads are lightweight Java threads that can be parked while waiting for many blocking operations. They improve scalability for I/O-heavy workloads, not CPU speed.

Follow the execution visually



Cons, caveats and common mistakes

1. Java 19/20 versions were previews; final in Java 21.
2. Do not pool virtual threads to throttle external services.
3. Use semaphores or connection limits for scarce resources.

Which approach should you choose?

Approach	Behavior	Good use case
platform threads	OS-backed; costlier	CPU work / existing apps
virtual threads	lightweight; many waits	I/O concurrency

LESSON 25 | JAVA 19 | Virtual threads first preview

Frequently asked and tricky interview questions - with reasoning

Q1. Do virtual threads make CPU work faster?

Answer and reasoning: No; they mainly help with many concurrent waits.

Q2. When finalized?

Answer and reasoning: Java 21.

Q3. Can you still use synchronized?

Answer and reasoning: Yes, but pinning behavior on JDK 21 needs care.

Practice: beginner -> intermediate -> advanced

Level 1: Run 100 sleep tasks with a fixed pool.

Level 2: Repeat with Java 21 virtual threads.

Level 3: Explain why database pool limits still matter.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 19 milestone: Virtual threads first preview. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Virtual threads preview Java 19

Lesson 25 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why does one OS thread per waiting request become expensive? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
ExecutorService pool =
    Executors.newFixedThreadPool(20);
```

AFTER: WHAT WE CAN WRITE

```
// Java 19 preview: virtual threads
// Java 21 final API:
Thread.startVirtualThread(task);
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Virtual threads are cheap to create, not free. Their main benefit is high-concurrency blocking I/O, not CPU speed.

INTERVIEW TRAP - PREDICT FIRST

Does 1000 virtual threads make a CPU loop 1000x faster?

ANSWER AND REASONING

No. CPU execution still uses available cores.

TRY IT / WORKED SOLUTION

Compare many sleep tasks with CPU loops. Answer: Waiting tasks benefit more than CPU-heavy work.

LESSON 26 | JAVA 19 | Record patterns first preview

Real-world problem, in plain language

A program receives nested records and needs to read their components. Repeated accessor calls obscure the data shape.

How developers approached it before

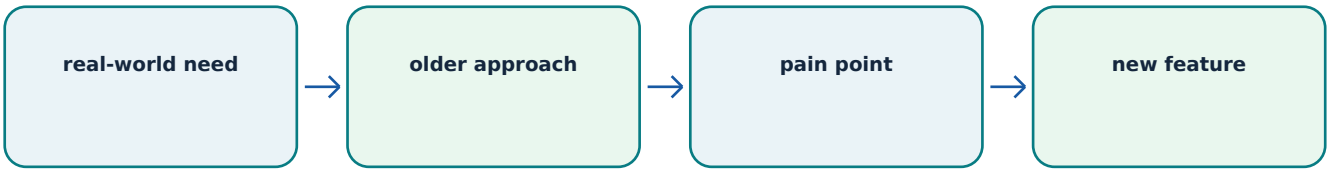
```
record Point(int x, int y) {}
if (obj instanceof Point p) {
  int x = p.x();
  int y = p.y();
}
```

What is inconvenient or risky?

Record patterns destructure a record: they test its shape and bind components directly.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 26 | JAVA 19 | Record patterns first preview

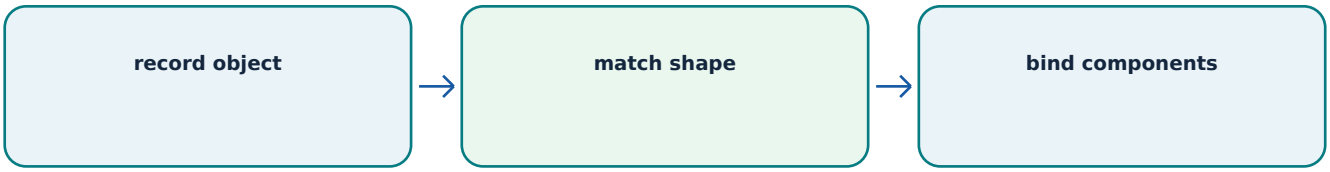
Modern approach (see version notes)

```
// Final in Java 21:
if (obj instanceof Point(int x, int y)) {
  System.out.println(x + y);
}
```

Why this helps

Record patterns destructure a record: they test its shape and bind components directly. The feature was previewed in Java 19/20 and finalized in Java 21.

Follow the execution visually



Cons, caveats and common mistakes

1. Use final syntax on Java 21.
2. Nested record patterns can become hard to read.
3. Patterns do not deep-copy data.

Which approach should you choose?

Approach	Behavior	Good use case
accessors	manual extraction	simple cases
record pattern	destructuring	nested data

LESSON 26 | JAVA 19 | Record patterns first preview

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Do record patterns create a new record?

Answer and reasoning: No; they read components from an existing record.

Q2. When finalized?

Answer and reasoning: Java 21.

Q3. Can a nested record pattern match nested records?

Answer and reasoning: Yes.

Practice: beginner -> intermediate -> advanced

Level 1: Write a Point pattern.

Level 2: Nest two records and destructure.

Level 3: Compare accessor calls with record pattern.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 19 milestone: Record patterns first preview. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Record patterns preview Java 19

Lesson 26 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can we unpack a record without repeated accessor calls? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
if (o instanceof Point p) {
 int x = p.x(); int y = p.y();
}
```

AFTER: WHAT WE CAN WRITE

```
if (o instanceof Point(int x, int y)) {
 System.out.println(x + y);
}
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Preview in Java 19/20; finalized in Java 21. Nested patterns can destructure nested records.

INTERVIEW TRAP - PREDICT FIRST

Do record patterns modify the record?

ANSWER AND REASONING

No. They read and bind component values.

TRY IT / WORKED SOLUTION

Try record Pair(Point left, Point right). Answer: Nested patterns can extract both coordinates.

LESSON 27 | JAVA 20 | Structured concurrency and scoped values previews

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A request launches two related tasks. Independent futures make cancellation and lifetime harder to coordinate. Passing request context via ThreadLocal can also be awkward.

How developers approached it before

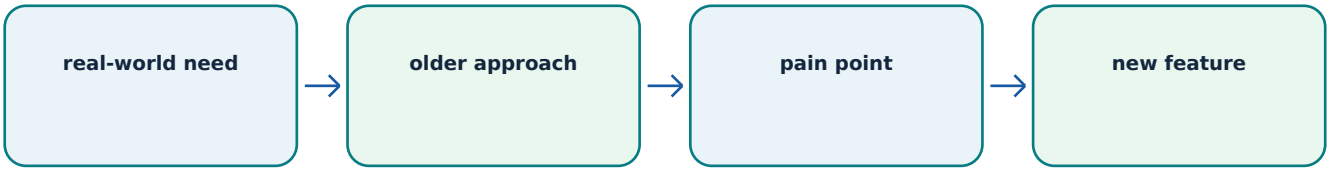
```
CompletableFuture<String> a =
    CompletableFuture.supplyAsync(() -> fetchA());
CompletableFuture<String> b =
    CompletableFuture.supplyAsync(() -> fetchB());
```

What is inconvenient or risky?

Structured concurrency groups child tasks under a parent scope; scoped values allow controlled sharing of immutable context.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 27 | JAVA 20 | Structured concurrency and scoped values previews

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

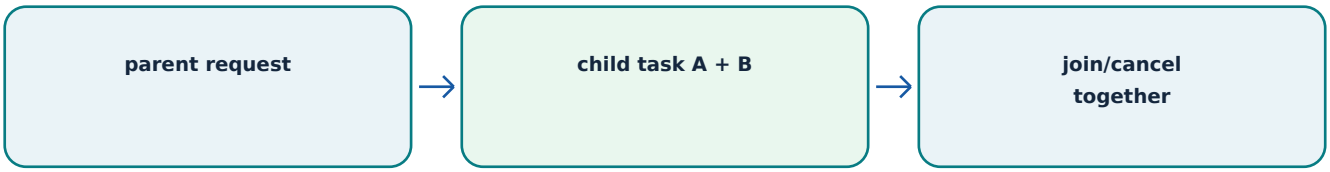
Modern approach (see version notes)

```
// Java 20: structured concurrency was incubating;
// scoped values were incubating (not final).
// APIs evolved after Java 20.
// Learn the idea: parent owns child tasks,
// request context has bounded lifetime.
```

Why this helps

Structured concurrency groups child tasks under a parent scope; scoped values allow controlled sharing of immutable context. In Java 20 these were evolving APIs, not stable production features.

Follow the execution visually



Cons, caveats and common mistakes

1. Do not copy a later preview API and label it Java 20.
2. Preview/incubator APIs can change across releases.
3. These features are not final Java 21 language APIs.

Which approach should you choose?

Approach	Behavior	Good use case
CompletableFuture	composable async API	independent tasks
structured concurrency	task lifetime as unit	related tasks

LESSON 27 | JAVA 20 | Structured concurrency and scoped values previews

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Were scoped values final in Java 20?

Answer and reasoning: No; incubating.

Q2. Why structure task lifetime?

Answer and reasoning: To manage completion, failure and cancellation together.

Q3. Are preview APIs stable contracts?

Answer and reasoning: No.

Practice: beginner -> intermediate -> advanced

Level 1: Sketch parent-child task lifecycle.

Level 2: Compare unstructured futures and scoped task ownership.

Level 3: Explain immutable request context.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 20 milestone: Structured concurrency and scoped values previews. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Structured concurrency / scoped values preview

Research Track: Structural awareness, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How can related tasks fail or cancel together? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
Future<A> a = pool.submit(this::loadA);
Future<B> b = pool.submit(this::loadB);
```

AFTER: WHAT WE CAN WRITE

```
// Java 21 preview: StructuredTaskScope
// Java 21 preview: ScopedValue
// APIs and flags are preview-specific.
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Preview APIs evolved after Java 21. Do not present them as stable Java 21 production APIs.

INTERVIEW TRAP - PREDICT FIRST

Why group child tasks?

ANSWER AND REASONING

A scope helps reason about lifetime, cancellation and errors together.

TRY IT / WORKED SOLUTION

Sketch parent -> child A and child B failure paths. Answer: A failed child can trigger policy-driven cancellation of siblings.

LESSON 28 | JAVA 20 | Virtual threads and record patterns second previews

Real-world problem, in plain language

Java 19 introduced previews; Java 20 incorporated feedback before finalization. Developers must not mistake preview availability for a stable contract.

How developers approached it before

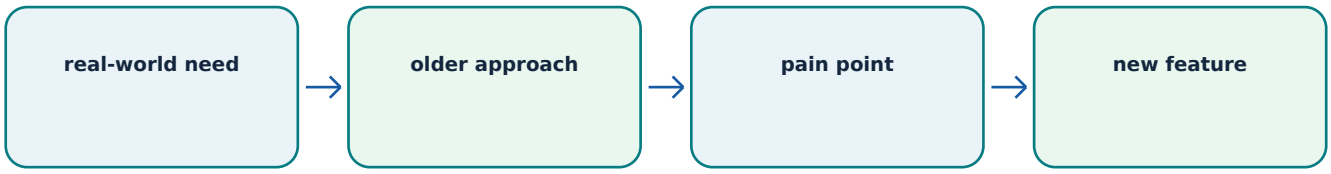
```
// Preview feature source can need flags:
// javac --enable-preview --release 20 ...
// java --enable-preview ...
```

What is inconvenient or risky?

A preview is a chance to test a language or API feature.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 28 | JAVA 20 | Virtual threads and record patterns second previews

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

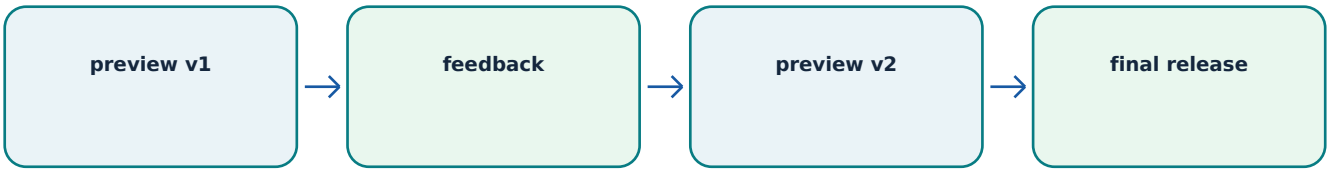
Modern approach (see version notes)

```
// Java 21 final APIs:
Thread.startVirtualThread(() -> doWork());
if (obj instanceof Point(int x, int y)) {
  System.out.println(x + y);
}
```

Why this helps

A preview is a chance to test a language or API feature. It may change before final release. Learn final Java 21 forms for production-oriented practice.

Follow the execution visually



Cons, caveats and common mistakes

1. Preview class files target specific JDK versions.
2. Do not assume preview code compiles without flags.
3. Structured concurrency remains preview in Java 21.

Which approach should you choose?

Approach	Behavior	Good use case
preview	subject to change	exploration
final	standard feature	production use

LESSON 28 | JAVA 20 | Virtual threads and record patterns second previews

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does preview mean deprecated?

Answer and reasoning: No; it means not yet finalized.

Q2. Why two previews?

Answer and reasoning: To refine behavior and API using feedback.

Q3. Is record pattern final in Java 20?

Answer and reasoning: No; final in Java 21.

Practice: beginner -> intermediate -> advanced

Level 1: Mark a version timeline for both features.

Level 2: Explain release vs preview status.

Level 3: Compile final syntax with --release 21.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 20 milestone: Virtual threads and record patterns second previews. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Virtual threads / record patterns second previews

Lesson 20 | Java 21 and Java 22 | reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why did Java repeat preview rounds? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Early preview behavior and syntax
// could change after feedback.
```

AFTER: WHAT WE CAN WRITE

```
// Use Java 21 final forms for
// virtual threads and record patterns.
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

A preview is not a promise that the same syntax will remain. Use --enable-preview only for matching JDK releases.

INTERVIEW TRAP - PREDICT FIRST

Can you compile Java 19 preview code unchanged on Java 21?

ANSWER AND REASONING

Not necessarily; preview APIs and syntax can evolve.

TRY IT / WORKED SOLUTION

Write a release timeline for the two features. Answer: Virtual threads: 19/20 preview, 21 final; record patterns: 19/20 preview, 21 final.

LESSON 29 | JAVA 21 | Virtual threads finalized

Real-world problem, in plain language

A service waits for 500 remote responses. A fixed pool of 20 platform threads runs only 20 blocking tasks at once. Virtual threads allow many more concurrent waits, subject to external resource limits.

How developers approached it before

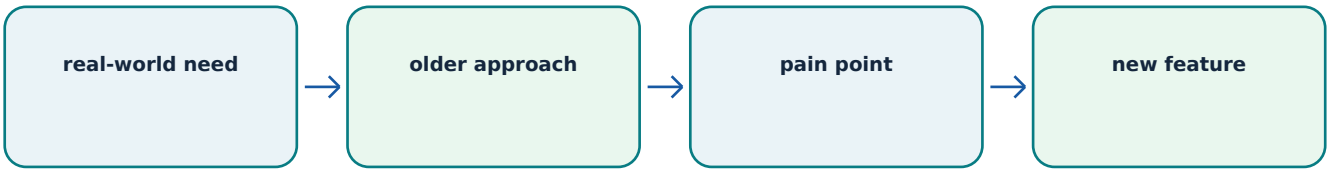
```
try (var pool = Executors.newFixedThreadPool(20)) {
  for (int i = 0; i < 500; i++) {
    pool.submit(() -> {
      try { Thread.sleep(100); }
      catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }
}
```

What is inconvenient or risky?

A virtual thread is still a Thread, but it is cheaper to create and can suspend during supported blocking operations.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 29 | JAVA 21 | Virtual threads finalized

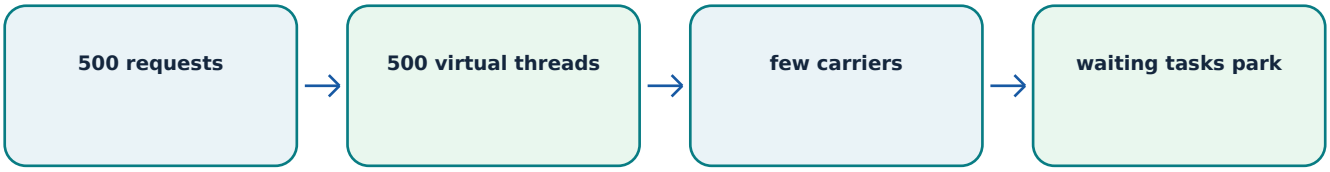
Modern approach (see version notes)

```
try (var pool =
    Executors.newVirtualThreadPerTaskExecutor()) {
  for (int i = 0; i < 500; i++) {
    pool.submit(() -> {
      try { Thread.sleep(100); }
      catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }
}
```

Why this helps

A virtual thread is still a Thread, but it is cheaper to create and can suspend during supported blocking operations. Carrier platform threads run virtual threads while they execute.

Follow the execution visually



Cons, caveats and common mistakes

1. Virtual threads do not remove database or API rate limits.
2. CPU-heavy loops still compete for CPU cores.
3. JDK 21 synchronized blocking can pin a carrier; measure before optimizing.

Which approach should you choose?

Approach	Behavior	Good use case
fixed pool	limits active platform threads	legacy workloads
virtual-per-task	many waiting tasks	I/O-bound services

LESSON 29 | JAVA 21 | Virtual threads finalized

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Should you pool virtual threads?

Answer and reasoning: Usually no; create per task and bound scarce resources separately.

Q2. Are virtual threads faster for sorting?

Answer and reasoning: Not inherently.

Q3. What is a carrier thread?

Answer and reasoning: A platform thread that executes a mounted virtual thread.

Practice: beginner -> intermediate -> advanced

Level 1: Measure 100 sleep tasks with fixed and virtual executors.

Level 2: Add Semaphore(10) around remote call.

Level 3: Explain pinning with a blocking synchronized section.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Virtual threads finalized. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Virtual threads final Java 21

Lesson 29 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we handle thousands of mostly waiting requests without huge platform-thread pools? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
try (var pool = Executors.newFixedThreadPool(20)) {
 // Java 8 uses shutdown instead of try-with-resources
}
```

AFTER: WHAT WE CAN WRITE

```
try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
 pool.submit(() -> fetchRemote());
}
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

ExecutorService AutoCloseable is modern; a Java 8 example must use shutdown. Limit database connections separately.

INTERVIEW TRAP - PREDICT FIRST

Should we pool virtual threads to 20?

ANSWER AND REASONING

Usually no. Create per task; limit scarce resources such as DB connections instead.

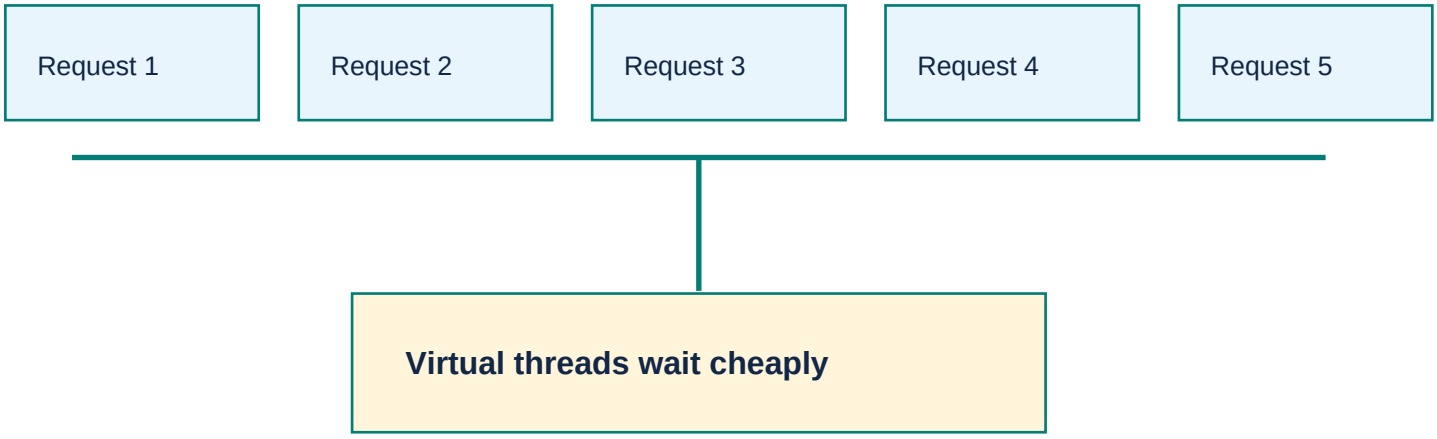
TRY IT / WORKED SOLUTION

Add a Semaphore(10) around DB work. Answer: This limits DB concurrency while allowing many virtual threads to wait.

THREAD LAB | Platform vs virtual threads

Read the arrows as references or execution steps; run the code before checking the answer.

REQUESTS WAITING FOR NETWORK / DATABASE



JAVA 21: ONE VIRTUAL THREAD PER TASK

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
  for (int i = 0; i < 100; i++) {
    ex.submit(() -> { Thread.sleep(100); return 1; });
  }
}
```

IMPORTANT LIMIT

Virtual threads do not increase CPU cores. A database pool with 10 connections still permits only about 10 simultaneous database operations.

INTERVIEW: WHY NOT POOL VIRTUAL THREADS?

They are designed to be created per task. Limit scarce resources (connections, rate limits) using pools or semaphores instead.

LESSON 30 | JAVA 21 | Pattern matching for switch finalized

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A service handles Success, Failure and Pending result objects. A chain of instanceof checks is repetitive and may miss a new case.

How developers approached it before

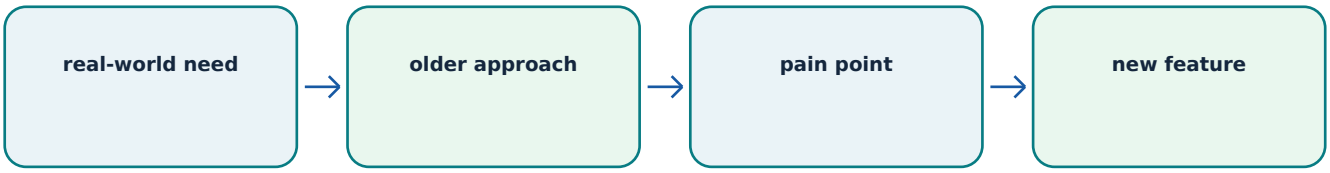
```
if (r instanceof Success) {
  System.out.println("ok");
} else if (r instanceof Failure) {
  System.out.println("failed");
}
```

What is inconvenient or risky?

A switch can match object types and bind variables.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 30 | JAVA 21 | Pattern matching for switch finalized

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

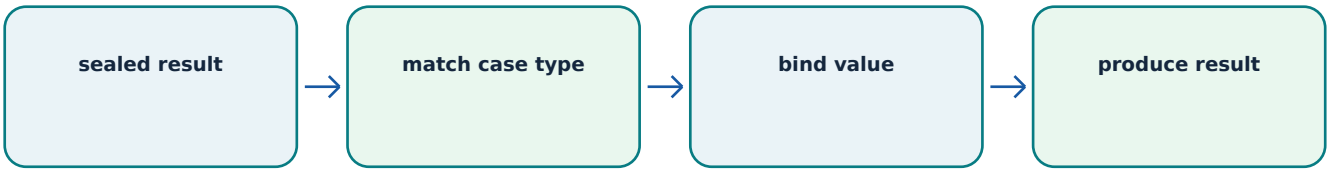
Modern approach (see version notes)

```
String message = switch (r) {
  case Success s -> "ok " + s.id();
  case Failure f -> "failed " + f.reason();
  case null -> "missing";
};
```

Why this helps

A switch can match object types and bind variables. With a sealed hierarchy, the compiler can check that all permitted alternatives are handled.

Follow the execution visually



Cons, caveats and common mistakes

1. Case ordering matters: broad patterns can dominate narrow ones.
2. Null handling must be intentional.
3. Exhaustiveness is a compile-time benefit with sealed hierarchies.

Which approach should you choose?

Approach	Behavior	Good use case
instanceof chain	manual branches	legacy
pattern switch	typed cases	closed domain models

LESSON 30 | JAVA 21 | Pattern matching for switch finalized

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Can switch match null in Java 21?

Answer and reasoning: Yes, using case null.

Q2. Why might a broad case be illegal before a narrow case?

Answer and reasoning: It dominates the narrow case, making it unreachable.

Q3. Why sealed helps?

Answer and reasoning: The compiler knows permitted variants.

Practice: beginner -> intermediate -> advanced

Level 1: Convert instanceof chain to switch.

Level 2: Add case null.

Level 3: Add a new permitted type and inspect exhaustiveness.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Pattern matching for switch finalized. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Pattern switch final Java 21

Lesson 30 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we branch on object types without chains of casts? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
if (o instanceof String) return ((String)o).length();
if (o instanceof Integer) return (Integer)o;
```

AFTER: WHAT WE CAN WRITE

```
return switch(o) {
 case String s -> s.length();
 case Integer i -> i;
 default -> 0;
};
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Order matters for dominated patterns; null handling is explicit with case null when needed.

INTERVIEW TRAP - PREDICT FIRST

Can case Object x appear before case String s?

ANSWER AND REASONING

No, the later String case is dominated/unreachable.

TRY IT / WORKED SOLUTION

Add case null -> -1. Answer: A switch on a null reference can handle null explicitly.

LESSON 31 | JAVA 21 | Record patterns finalized

Real-world problem, in plain language

Nested API responses contain a customer and address. Multiple accessor calls are repetitive, especially inside pattern switches.

How developers approached it before

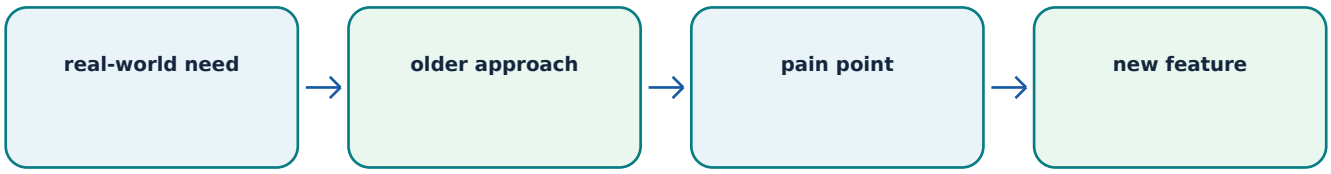
```
record Address(String city) {}
record Customer(Address address) {}
Customer c = new Customer(new Address("Pune"));
System.out.println(c.address().city());
```

What is inconvenient or risky?

Record patterns match and unpack record components in a single expression.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 31 | JAVA 21 | Record patterns finalized

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

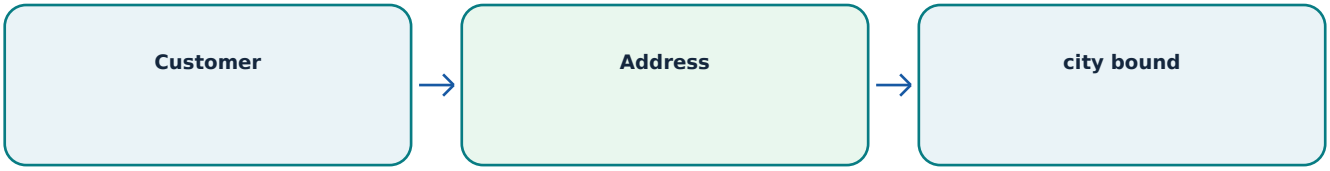
Modern approach (see version notes)

```
record Address(String city) {}
record Customer(Address address) {}
Object value = new Customer(new Address("Pune"));
if (value instanceof Customer(Address(String city))) {
  System.out.println(city);
}
```

Why this helps

Record patterns match and unpack record components in a single expression. The nested pattern reveals the expected data shape.

Follow the execution visually



Cons, caveats and common mistakes

1. Nested patterns can reduce readability if deeply nested.
2. Record patterns do not make mutable fields immutable.
3. Null values may prevent a nested pattern match.

Which approach should you choose?

Approach	Behavior	Good use case
accessors	direct navigation	simple model
record patterns	shape-based matching	nested variants

LESSON 31 | JAVA 21 | Record patterns finalized

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does destructuring allocate copies?

Answer and reasoning: No.

Q2. What if address is null?

Answer and reasoning: The nested Address pattern does not match.

Q3. Why not always use patterns?

Answer and reasoning: Simple accessor code may be clearer.

Practice: beginner -> intermediate -> advanced

Level 1: Create nested record patterns.

Level 2: Predict null nested component behavior.

Level 3: Compare pattern and accessor readability.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Record patterns finalized. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Record patterns final Java 21

Lesson 31 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we destructure nested data in one readable check? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
if (obj instanceof Point p) {
 System.out.println(p.x());
}
```

AFTER: WHAT WE CAN WRITE

```
if (obj instanceof Point(int x, int y)) {
 System.out.println(x + y);
}
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Record patterns bind components but do not validate domain rules by themselves.

INTERVIEW TRAP - PREDICT FIRST

Are component objects copied by record patterns?

ANSWER AND REASONING

No. References are bound to existing component objects.

TRY IT / WORKED SOLUTION

Try a nested record and two levels of patterns. Answer: Nested record patterns remove repeated accessor calls.

LESSON 32 | JAVA 21 | Sequenced collections

Real-world problem, in plain language

A program needs the first and last element of a list, and wants one vocabulary for ordered lists, sets and maps.

How developers approached it before

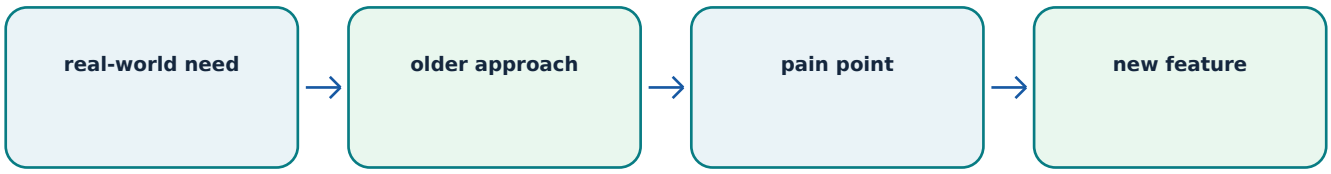
```
List<String> xs = new ArrayList<>();
xs.add("a"); xs.add("b");
String first = xs.get(0);
String last = xs.get(xs.size() - 1);
```

What is inconvenient or risky?

SequencedCollection adds first/last/reversed operations.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 32 | JAVA 21 | Sequenced collections

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

Modern approach (see version notes)

```
List<String> xs = new ArrayList<>(List.of("a","b"));
System.out.println(xs.getFirst());
System.out.println(xs.getLast());
System.out.println(xs.reversed());
```

Why this helps

SequencedCollection adds first/last/reversed operations. SequencedSet and SequencedMap extend the idea to collections with defined encounter order.

Follow the execution visually



Cons, caveats and common mistakes

1. reversed() is a view for many implementations, not necessarily a copy.
2. Not every Set has defined encounter order.
3. getFirst/getLast on an empty collection throws.

Which approach should you choose?

Approach	Behavior	Good use case
get(0)	list-specific	legacy
getFirst	sequenced vocabulary	Java 21

LESSON 32 | JAVA 21 | Sequenced collections

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does reversed() always copy?

Answer and reasoning: No; often it is a reverse-order view.

Q2. Does HashSet have reliable encounter order?

Answer and reasoning: No.

Q3. What happens on empty getFirst()?

Answer and reasoning: NoSuchElementException.

Practice: beginner -> intermediate -> advanced

Level 1: Try ArrayList.reversed and modify original.

Level 2: Compare LinkedHashSet and HashSet order.

Level 3: Use getFirst and getLast on empty list.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Sequenced collections. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Sequenced collections

Lesson 32 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why was first/last access inconsistent across collection types? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
list.get(0);
list.get(list.size()-1);
```

AFTER: WHAT WE CAN WRITE

```
list.getFirst();
list.getLast();
var backwards = list.reversed();
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

SequencedCollection provides a common order-aware API. reversed() is a view, not necessarily a copy.

INTERVIEW TRAP - PREDICT FIRST

If original changes, can reversed view change?

ANSWER AND REASONING

Yes, for mutable backing collections it reflects changes.

TRY IT / WORKED SOLUTION

Reverse a mutable ArrayList and change the source. Answer: The reversed view tracks the backing collection.

LESSON 33 | JAVA 21 | Generational ZGC and garbage collection

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

An application allocates many short-lived request objects. The JVM must reclaim unreachable objects while keeping response pauses low.

How developers approached it before

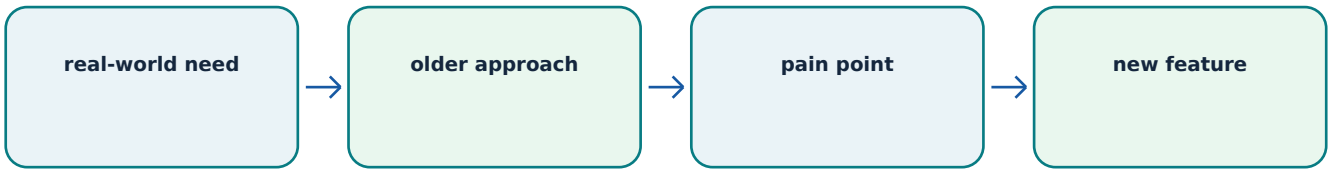
```
Object p = new Object();
p = null; // old object may become unreachable
// GC runs when JVM decides, not immediately.
```

What is inconvenient or risky?

Generational collection uses the observation that many objects die young.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 33 | JAVA 21 | Generational ZGC and garbage collection

PAGE 2 OF 3 / NEW CODE + HOW IT WORKS

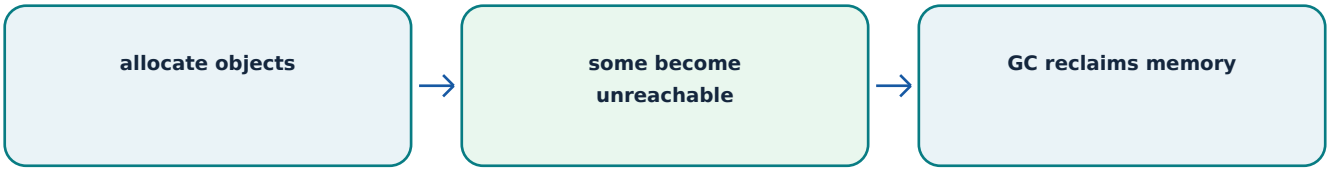
Modern approach (see version notes)

```
// Java 21: Generational ZGC available.
// Example JVM options (environment dependent):
// -XX:+UseZGC -XX:+ZGenerational
// java -Xlog:gc* ...
```

Why this helps

Generational collection uses the observation that many objects die young. ZGC aims for low pause times; Java 21 introduced its generational mode. Reachability, not setting a variable to null alone, determines eligibility.

Follow the execution visually



Cons, caveats and common mistakes

1. GC does not run immediately when a reference is cleared.
2. Low pauses do not mean zero cost.
3. Measure latency, throughput and memory before choosing collector.

Which approach should you choose?

Approach	Behavior	Good use case
G1	balanced default collector	general workloads
ZGC	low-pause focus	latency-sensitive workloads

LESSON 33 | JAVA 21 | Generational ZGC and garbage collection

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does p=null immediately free memory?

Answer and reasoning: No; it can make an object eligible if no other references remain.

Q2. Does GC always improve performance?

Answer and reasoning: No; collector choice has tradeoffs.

Q3. What is a GC root?

Answer and reasoning: A starting reference from which reachable objects are traced.

Practice: beginner -> intermediate -> advanced

Level 1: Draw two references to same object and remove one.

Level 2: Enable GC logs and inspect events.

Level 3: Compare throughput vs pause-time goals.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Generational ZGC and garbage collection. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Generational ZGC

Lesson 33 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why separate short-lived objects from long-lived objects? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Object allocation and reachability
byte[] temp = new byte[1024];
```

AFTER: WHAT WE CAN WRITE

```
// Java 21: -XX:+UseZGC
// -XX:+ZGenerational
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Generational ZGC in Java 21 is optional and experimental-era configuration; measure actual latency/throughput.

INTERVIEW TRAP - PREDICT FIRST

Does setting a variable to null immediately free memory?

ANSWER AND REASONING

No. Collection happens later, and reachability is the real criterion.

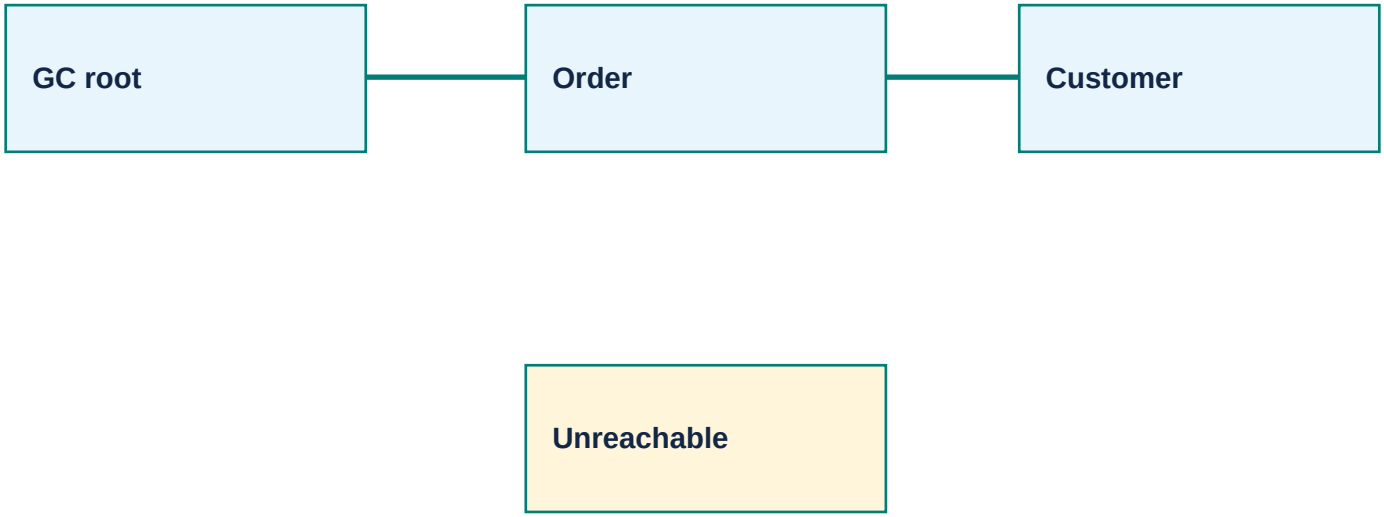
TRY IT / WORKED SOLUTION

Run -Xlog:gc* on a sample allocation loop. Answer: Look for collection events and pauses; do not infer performance from one run.

JVM LAB | Reachability and garbage collection

Read the arrows as references or execution steps; run the code before checking the answer.

REACHABLE OBJECTS STAY ALIVE



CODE: ELIGIBLE IS NOT IMMEDIATELY COLLECTED

```
Object x = new Object();
x = null;  // old object may become unreachable
// No guarantee when GC will reclaim it.
```

GENERATIONS: A PERFORMANCE IDEA

Many objects die young; some live much longer. Generational collectors use this observation. Java 21 added a generational mode for ZGC.

RUN AND OBSERVE

```
java -Xlog:gc* -XX:+UseZGC -XX:+ZGenerational App
Compare logs under the same workload. Do not assume one collector wins everywhere.
```

LESSON 34 | JAVA 21 | Java 21 preview features

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A team sees examples of string templates, structured concurrency, scoped values and unnamed patterns online. Not all are stable Java 21 APIs.

How developers approached it before

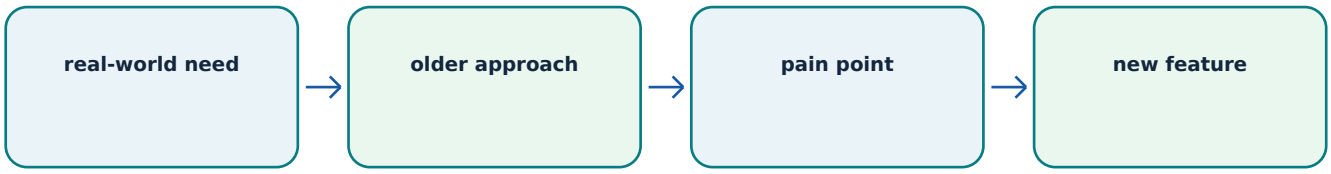
```
// Java 8-style concatenation:
String message = "Hello " + name;
// CompletableFuture for concurrent tasks.
```

What is inconvenient or risky?

Version status matters.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 34 | JAVA 21 | Java 21 preview features

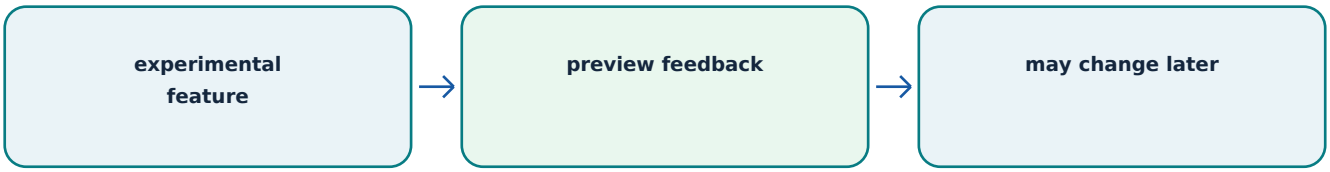
Modern approach (see version notes)

```
// Java 21 preview/incubator status:
// String templates: preview (later withdrawn)
// Structured concurrency: preview
// Scoped values: preview
// Unnamed patterns/variables: preview
```

Why this helps

Version status matters. Preview features may require --enable-preview and can change or disappear. Prefer finalized Java 21 features for a stable learning baseline.

Follow the execution visually



Cons, caveats and common mistakes

1. String templates did not become a stable Java 21 feature.
2. Preview flags must match JDK version.
3. Avoid teaching experimental syntax as finalized.

Which approach should you choose?

Approach	Behavior	Good use case
final	standard	safe baseline
preview	experimental	learning/evaluation

LESSON 34 | JAVA 21 | Java 21 preview features

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Are string templates final in Java 21?

Answer and reasoning: No; preview only, later withdrawn.

Q2. Are virtual threads preview in Java 21?

Answer and reasoning: No; finalized.

Q3. Do preview APIs always survive unchanged?

Answer and reasoning: No.

Practice: beginner -> intermediate -> advanced

Level 1: Label every feature as final/preview/incubator.

Level 2: Explain why experimental syntax can break upgrades.

Level 3: Use stable alternatives for production examples.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Java 21 preview features. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Java 21 preview features

Lesson 34 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

How do we experiment without confusing preview APIs with stable APIs? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Stable Java 21 code needs no preview flag.
```

AFTER: WHAT WE CAN WRITE

```
// Preview compilation: javac --enable-preview
// --release 21 Example.java
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Preview APIs may change. Match preview flags to the exact release; avoid using previews casually in production.

INTERVIEW TRAP - PREDICT FIRST

Does Java 21 preview mean finalized in Java 21?

ANSWER AND REASONING

No. Preview explicitly means not finalized.

TRY IT / WORKED SOLUTION

Compile a preview sample with and without flags. Answer: The compiler rejects preview use unless enabled.

LESSON 35 | JAVA 21 | Migration checklist and compatibility

PAGE 1 OF 3 / WHY THIS FEATURE EXISTS

Real-world problem, in plain language

A production Java 8 service must move to Java 21. Language syntax is only one part: dependencies, removed modules, reflection, build tools and tests can break.

How developers approached it before

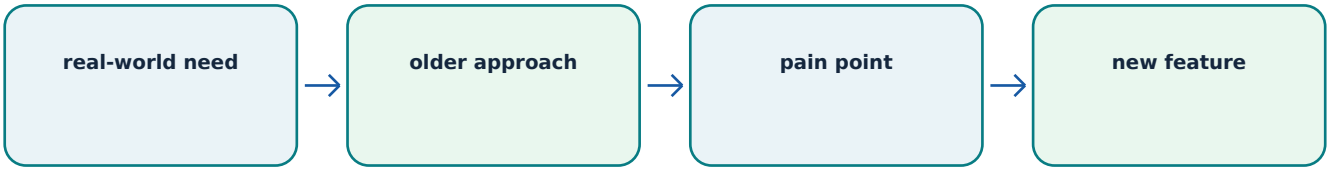
```
// Java 8 app may rely on JAXB bundled in JDK,
// older libraries, or internal reflective access.
// Compilation success alone is insufficient.
```

What is inconvenient or risky?

A safe upgrade inventories dependencies, compiles, runs tests, verifies runtime behavior, and measures performance.

Think before moving on

Look at the old code. Which line is doing the work? Which line is only ceremony? What could go wrong if a second developer changes the input? Write your prediction before turning the page.



Vocabulary checkpoint

If you meet a new term, first translate it into everyday language. Ask: is it a new object, a new reference, a new syntax rule, or a runtime behavior?

LESSON 35 | JAVA 21 | Migration checklist and compatibility

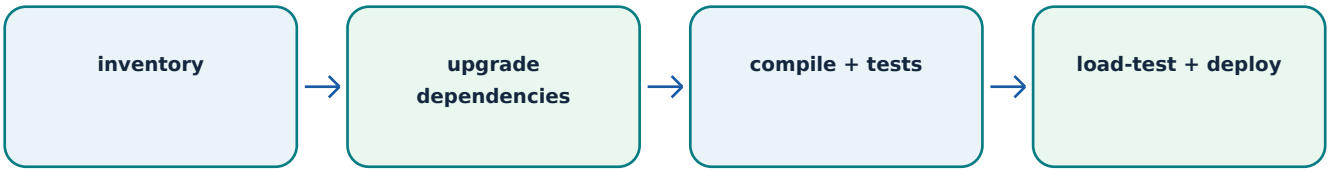
Modern approach (see version notes)

```
// Typical migration commands:
// java -version
// javac --release 21 Main.java
// jdeps --jdk-internals myapp.jar
// java -Xlog:gc* -jar myapp.jar
```

Why this helps

A safe upgrade inventories dependencies, compiles, runs tests, verifies runtime behavior, and measures performance. Keep migration distinct from adopting every new language feature.

Follow the execution visually



Cons, caveats and common mistakes

1. JAXB/JAX-WS were removed from JDK 11; add dependencies as needed.
2. Strong encapsulation can affect old reflection.
3. Do not change JVM version and architecture blindly in one step.

Which approach should you choose?

Approach	Behavior	Good use case
compile	syntax/API checks	first gate
tests	behavior checks	second gate
load test	performance checks	deployment gate

LESSON 35 | JAVA 21 | Migration checklist and compatibility

PAGE 3 OF 3 / INTERVIEW LAB + PRACTICE

Frequently asked and tricky interview questions - with reasoning

Q1. Does compiling prove runtime compatibility?

Answer and reasoning: No; reflection and configuration can fail at runtime.

Q2. What does --release control?

Answer and reasoning: Source language, bytecode target and exposed platform APIs.

Q3. Should all Java 21 features be adopted immediately?

Answer and reasoning: No; migrate safely, then refactor selectively.

Practice: beginner -> intermediate -> advanced

Level 1: Create a dependency checklist.

Level 2: Run jdeps on a sample jar.

Level 3: Draft rollback and performance validation plan.

Your explanation checklist

Can I explain (1) the problem, (2) the older code, (3) the new code, (4) one caveat, and (5) one real-world use case without reading the page? If not, repeat the smallest failing example.

Remember

Java 21 milestone: Migration checklist and compatibility. Do not confuse an API convenience with a deep change in object behavior. Test the exact contract and release status.

EXPLAINED FURTHER | Migration and compatibility

Lesson 35 | Extra examples, reasoning, and worked answers

THE SITUATION: WHY SHOULD YOU CARE?

Why can a Java 8 app compile but fail on Java 21? Imagine this is real application code: what can go wrong as the data or traffic grows?

BEFORE: WHAT WE WROTE

```
// Java 8 app may depend on JAXB
// or internal JDK reflection.
```

AFTER: WHAT WE CAN WRITE

```
// Upgrade dependencies; add explicit JAXB libs.
// Test on JDK 21 with warnings and GC logs.
```

CAVEAT: WHAT THE NEW FEATURE DOES NOT SOLVE

Migration is a dependency and runtime problem, not just a language syntax upgrade.

INTERVIEW TRAP - PREDICT FIRST

Is setting --release 8 enough to run on Java 8?

ANSWER AND REASONING

It limits compilation APIs and bytecode but dependencies and runtime behavior also matter.

TRY IT / WORKED SOLUTION

Audit dependencies and run unit/integration tests. Answer: Look for removed modules, reflective access, charset assumptions, and JVM flags.

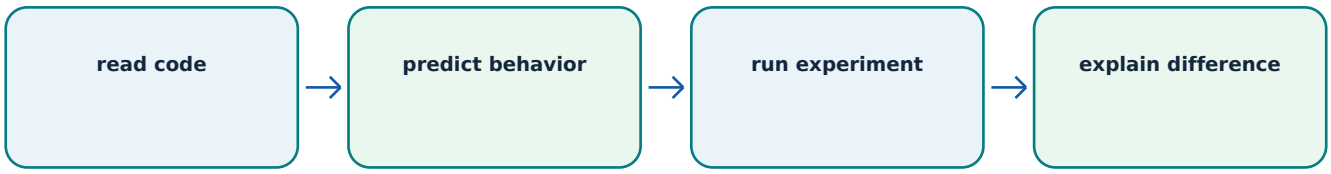
DEEP-DIVE LAB 1: Collection decision lab

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
String[] arr = {"A", "B"};
List<String> backed = Arrays.asList(arr);
List<String> copy = new ArrayList<>(backed);
List<String> readOnly = Collections.unmodifiableList(copy);
List<String> snapshot = List.copyOf(copy);
arr[0] = "Z";
copy.set(1, "Y");
System.out.println(backed);
System.out.println(readOnly);
System.out.println(snapshot);
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

backed=[Z,B]; readOnly=[A,Y]; snapshot=[A,B]. The backing array affects backed, the mutable copy affects its live wrapper, and copyOf preserves its membership.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

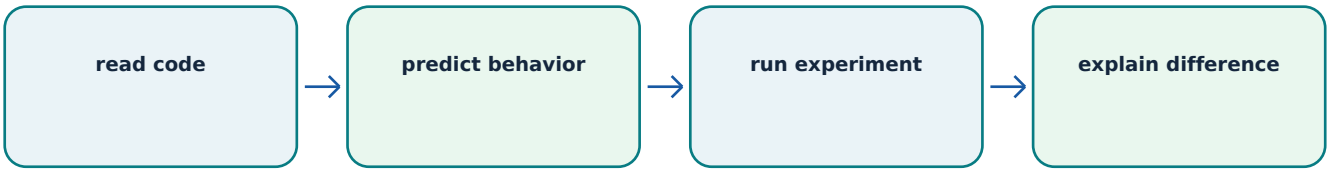
DEEP-DIVE LAB 2: Mutable objects inside snapshots

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
StringBuilder sb = new StringBuilder("Java");
List<StringBuilder> src = new ArrayList<>();
src.add(sb);
List<StringBuilder> snap = List.copyOf(src);
sb.append(" 21");
src.add(new StringBuilder("Go"));
System.out.println(snap.get(0));
System.out.println(snap.size());
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

Java 21 and 1. The same StringBuilder is shared, but new membership is not copied later.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

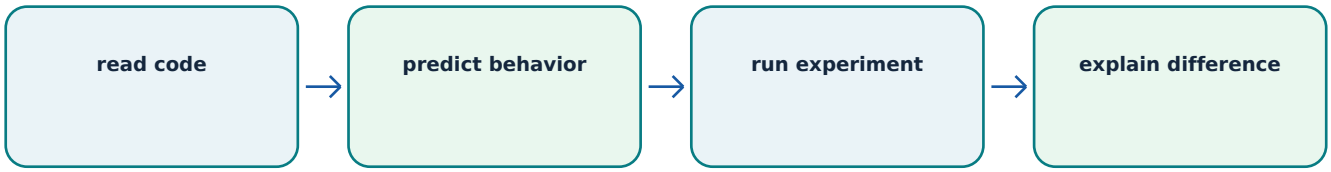
DEEP-DIVE LAB 3: Stream prefix trap

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
List<Integer> nums = List.of(1, 2, 9, 3);
System.out.println(nums.stream()
  .filter(n -> n < 5).toList());
System.out.println(nums.stream()
  .takeWhile(n -> n < 5).toList());
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

[1,2,3] then [1,2]. filter examines all elements; takeWhile stops at the first failure.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

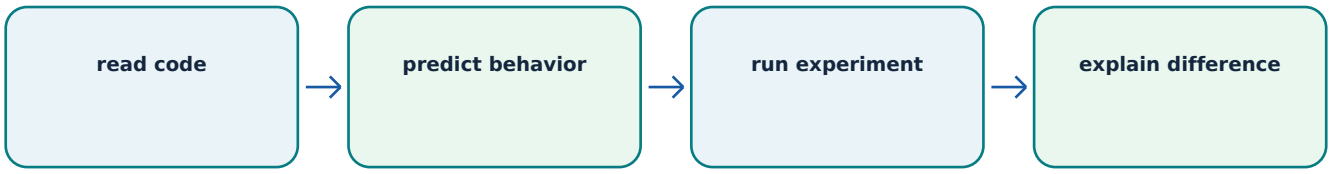
DEEP-DIVE LAB 4: Records and defensive copies

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
record Basket(List<String> items) {
  Basket {
    items = List.copyOf(items);
  }
}
List<String> input = new ArrayList<>(List.of("A"));
Basket b = new Basket(input);
input.add("B");
System.out.println(b.items());
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

[A]. The compact constructor copies membership. Without it, the record field is final but the list can remain mutable.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

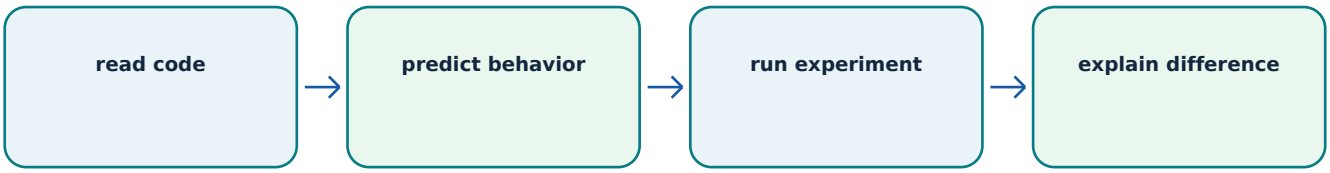
DEEP-DIVE LAB 5: Virtual threads and limits

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
Semaphore permits = new Semaphore(5);
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
  for (int i = 0; i < 50; i++) {
    ex.submit(() -> {
      permits.acquireUninterruptibly();
      try { Thread.sleep(100); }
      catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      } finally { permits.release(); }
    });
  }
}
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

50 virtual threads can exist, but at most 5 tasks hold a permit. A semaphore limits use of scarce external resources; it is not a thread pool.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

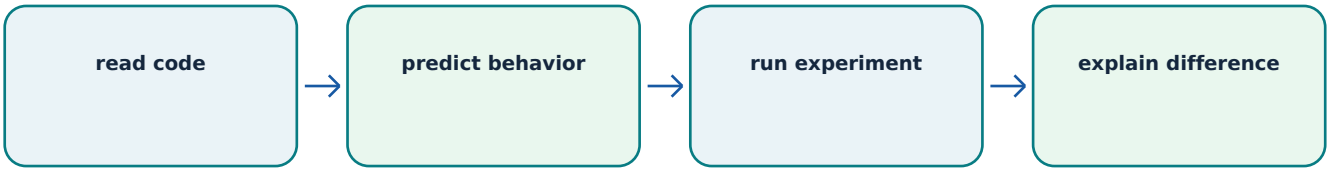
DEEP-DIVE LAB 6: Module visibility troubleshooting

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
// payment.api/module-info.java
module payment.api { exports payment.api; }
// payment.app/module-info.java
module payment.app { requires payment.api; }
// remove exports: public Payment becomes inaccessible
// to payment.app despite being public.
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

public on a class does not override module export boundaries. requires declares dependency; exports declares accessible packages.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

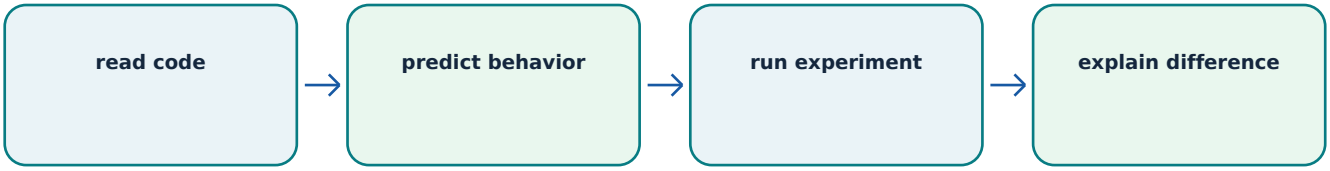
DEEP-DIVE LAB 7: Garbage collector reachability

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
class Node { Node next; }
Node a = new Node();
Node b = new Node();
a.next = b;
Node c = a;
a = null;
// both nodes remain reachable via c
c = null;
// if no other roots reference them, eligible for GC
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

A reachable root can keep an entire chain alive. Setting a variable to null does not guarantee immediate collection.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

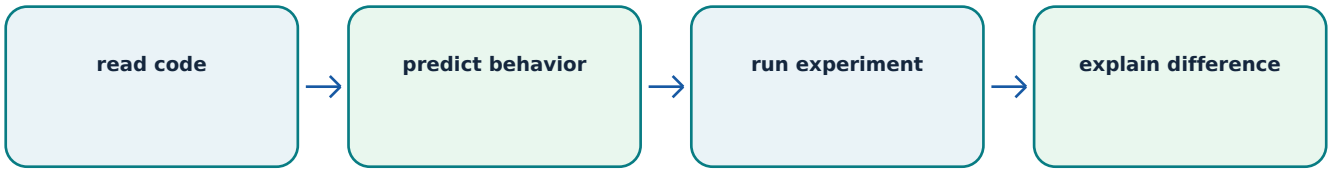
DEEP-DIVE LAB 8: Migration failure investigation

A longer practical case. Read the code without running it. Write the output and explain every object, reference or runtime step involved.

```
// 1. javac --release 21 ...
// 2. jdeps --jdk-internals app.jar
// 3. run tests and integration tests
// 4. measure GC and latency
// 5. check reflective-access failures
```

Predict before revealing

What is printed, what fails, or what behavior changes? Draw a small memory or execution diagram. Try to identify at least one incorrect assumption a beginner might make.



Reasoned solution

Compatibility includes compilation, dependencies, runtime reflection, removed JDK modules, external services and performance.

Interview follow-ups

What would change if the input were null? What if the element object were mutable? Would the same answer hold on Java 8? Which exact Java release first supports this syntax?

Extend the exercise

Make one controlled modification. Before running it, state whether the change affects compilation, collection membership, object state, or runtime performance.

WORKED EXERCISES | Predict, explain, then verify

Read the arrows as references or execution steps; run the code before checking the answer.

1. LIVE VIEW OR SNAPSHOT?

```
List<String> a = new ArrayList<>(List.of("A"));
var view = Collections.unmodifiableList(a);
var copy = List.copyOf(a); a.add("B");
// view=[A,B], copy=[A]
```

2. ORDERED STREAM TRAP

```
List.of(1,2,8,3).stream().takeWhile(n -> n < 5)
// [1,2], not [1,2,3]. filter would include 3.
```

3. RECORD SHALLOW IMMUTABILITY

```
record Box(List<String> items) {}
var xs = new ArrayList<>(List.of("A"));
var box = new Box(xs); xs.add("B");
// box.items() now contains B.
```

4. VIRTUAL THREADS AND CPU

```
Running 100 CPU-heavy tasks in 100 virtual threads
does not create 100 CPU cores. Test I/O waits separately.
```

Final revision: choose the right construct

Need	Use	Avoid confusing with
Editable independent list	new ArrayList<>(source)	same-reference alias
Read-only live view	Collections.unmodifiableList(source)	snapshot
Unmodifiable snapshot	List.copyOf(source)	deep copy
Known fixed elements	List.of(a,b)	mutable list
Concise DTO	record	deep immutability
Closed type family	sealed	final fields
Many I/O waits	virtual threads	faster CPU calculations
Strong dependency boundaries	JPMS	package alone

Final interview rehearsal

Explain with runnable code: Why does final List not prevent add()?

Explain with runnable code: How do shallow copies differ from deep copies?

Explain with runnable code: Why does Arrays.asList(array) reflect array changes?

Explain with runnable code: Why does List.copyOf not protect mutable elements?

Explain with runnable code: When is takeWhile different from filter?

Explain with runnable code: Why is var still statically typed?

Explain with runnable code: When should you choose a record instead of a class?

Explain with runnable code: What makes sealed types useful with pattern switch?

Explain with runnable code: Why do virtual threads not accelerate CPU-bound work?

Explain with runnable code: How do requires, exports and opens differ?

Explain with runnable code: What is reachability in garbage collection?

Explain with runnable code: What can break when migrating Java 8 to Java 21?

Release history and technical references

The book uses Java 21 as the practice baseline and distinguishes preview/incubator milestones from finalized features. Confirm API availability when targeting earlier releases. Primary references:

OpenJDK JEP index: https://openjdk.org/jeps/0

Java 21 language changes:
https://docs.oracle.com/en/java/javase/21/language/java-language-changes-summary.html

Java 21 migration guide: https://docs.oracle.com/en/java/javase/21/migrate/

Java learning portal: https://dev.java/learn/

JEP 444 (Virtual Threads), JEP 441 (Pattern Matching for switch), JEP 440 (Record Patterns), JEP 431 (Sequenced Collections), JEP 439 (Generational ZGC).

Quality and scope note

Every version from Java 9 to 21 is represented. Each lesson contains a specific use case, older and newer code, a flow diagram, caveats, comparison, three interview questions with explained answers, and three exercises. Longer labs revisit the highest-risk misunderstandings. Illustrative snippets may require surrounding imports/classes; preview examples are intentionally labelled.

This is a teaching workbook, not an exhaustive listing of every JEP. API and language release status is prioritized over sheer page count.